

TM470 – COMPUTING AND IT (BROAD – COMPUTER SCIENCE)

Digitalisation: Routine maintenance inspection
forms in a light-industry factory setting

Extended modules

TM352 – Web, mobile and cloud technologies

TM354 – Software Engineering

Lukasz Brzozowski

E5937290

Contents Page (with page numbers)

Contents Page (with page numbers)	1
Main Body of Report.....	3
Project Goals, Description, Scope and Impact.....	3
Account of Related Literature, including Review of Sources.....	5
Resources, Planning, Risks and Lifecycle Review.....	7
Project Work: Analysing and Solving a Practical Problem, demonstrating how your Computing and IT knowledge and skills have been extended	14
LSEPI and EDI Concerns.....	30
Personal Development, Review and Reflection.....	32
References	34
Bibliography	37
Epilogue.....	37
Appendix 1 – LSDEPI Forms	37
EMA LSEPI Form	39
Invitation paragraph	40
General information about the research study and collected research data	41
What will I be asked to do if I agree to take part?	41
How will the data I provide be used?	41
Your right to withdraw from the study	42
How do I agree to take part?	42
Thank you	42
INFORMED CONSENT FOR Developing a Digital Maintenance Inspection System	43
1. Taking part in the study	44
2. Use of the information in the study	45
3. Future use and reuse of the information by others	46
4. Signatures	47
Appendix 2 – Project Log Extracts.....	47
Appendix 3 – Stakeholders' input	49
Appendix 4 – Glossary of domain terms.....	53
Appendix 4 – PROMPT Checklist.....	54
Presentation.....	54
Relevance	54
Objectivity	54

Method: for research reports	54
Provenance	54
Timeliness	54
Appendix 5 – Evolution of classes, attributes and domain structural models	59
Appendix 6 – Supplementary system diagrams.....	66
Appendix 7 – Elaborated use cases.....	67
Appendix 8 – Functional and non-functional requirements.....	69
Appendix 9 – Wireframes – screenshots	72
.....	76
Appendix 10 –	76
Further Appendices.....	80

Main Body of Report

Project Goals, Description, Scope and Impact

1.1 Problem

The main problem is relying on paper forms for routine maintenance checks in factory environments. Besides, handling paper forms can be challenging in confined or poorly lit areas, leading to inefficiencies caused by manual errors from handwriting or misreading (SFG20, 2024). Furthermore, the paper-based system complicates data archiving and audit preparation (BSC, 2023), resulting in inefficiencies and missed opportunities for trend analysis. Additionally, the lack of real-time data entry holds back timely decision-making and effective resolution of recurring faults.

1.2 Purpose of the project

The purpose of this project is to design and develop a prototype of a digital inspection system that replaces the inefficient current process. This system aims to enable engineers to perform inspections digitally in real-time, reducing administrative burdens and enhancing consistency. It also seeks to improve data accuracy, traceability, and regulatory compliance while streamlining inspections. For managers, this solution would generate searchable records for trend analysis and audit preparation, ultimately increasing operational efficiency.

1.3 Importance of addressing the problem

According to PUWER (1998), equipment must be thoroughly checked by competent persons to prevent dangerous deterioration (HSE, 1998). Inspections must be documented and accessible per Regulation 6.3 of PUWER, which emphasises record-keeping. Current impractical solutions highlight the need for a more reliable data storage system.

Lacking digital infrastructure limits monitoring maintenance across departments or shifts. It impedes visibility of inspection schedules, technician accountability, and issue tracking. As operations rely more on data-driven decisions (IEEE, 2022), manual forms become outdated, reducing efficiency and accuracy (Conroy, 2023). Addressing this is crucial for maintaining oversight, integrity, and governance.

1.4 Benefits of addressing the problem

Enhancing the paper-based inspection system aims to enable quicker, more dependable data collection (Alleva, 2024), allowing engineers to fill forms in real-time and focus on technical tasks instead of paperwork. Digitisation also aims to improve traceability and audit readiness, making past records easy to access for audits or investigating recurring issues (SFG20, 2024; HSE, 1998). Access to historical data should become easier, supporting trend analysis and better decision-making on maintenance and equipment reliability (Glynn, 2023). Attaching photographic evidence should improve documentation by clarifying fault identification, especially for failed checks or hazards. The commercial version of the application should provide managers instant access to inspection data, enhancing responsiveness and corrective actions. From a sustainability view, reducing paper and ink use aligns with environmental goals. British Computing Society (2023) states that switching to digital forms reduces waste and resource use, promoting a more environmentally responsible workplace, aligning with the organisation's sustainability goals (GWP, 2024).

1.5 Potential solution

The proposed solution should replicate the current manual process and adhere to the standard currently in use. The solution should support two types of users: the engineers and the managers. The engineers should benefit from an improved user experience that allows them to fill in their structured input in an intuitive manner. The use of a digital template should guide the input of comments, field entries, and photographic evidence. Managers should review earlier inspection findings in a structured and intuitive manner. The solution is expected to reduce errors, improve incident and fault documentation, enhance the efficiency of maintenance inspections, and empower managers to search historical inspection evidence more effectively.

1.6 Project goals and deliverables

The proposed system is a prototype designed to enable engineers to carry out at least one maintenance inspection for each type: Facilities or Machine Safety. It includes a simple database for storing and retrieving user-entered data, along with a user-friendly interface that makes completing inspection forms easier. Furthermore, the proposed solution supports role-based access, a key security feature common in modern software solutions, ensuring that only authorised users can access the data.

From an academic perspective, the project employs the TM354 software engineering methods, including requirements elicitation, domain modelling, and iterative system development. It also investigates and compares various architectural options, such

as web-based versus desktop solutions, using feasibility analysis and stakeholder feedback. The project additionally produces a hybrid application prototype that applies and extends the knowledge presented by the TM352 web, mobile, and cloud technologies as a blueprint for a commercial version of the software, which will be built later, beyond the TM470.

1.7 Impact

The digitisation of standard inspections is anticipated to provide considerable efficiency gains. Engineers are expected to spend less time on administrative tasks and focus more on valuable activities, while managers will benefit from organised and searchable records of inspections. Utilising mobile devices is forecasted to remove delays that come from handling, scanning, and uploading paper documents. The effects extend beyond just time savings. Secure storage, access based on roles, and the ability to add attachments should enhance the quality of documentation, foster greater accountability, and strengthen the organisation's compliance position. Furthermore, swapping out paper forms aligns with the organisation's sustainability goals by reducing material waste and printing expenses.

Although this academic submission centres on the prototype, it also documents the development process, domain modelling, and database design, allowing for a complete production system to be built in the future.

Account of Related Literature, including Review of Sources

A strong theoretical foundation was established through Sommerville's *Software Engineering* (2016), which provided a structured approach to requirements engineering, iterative design, and socio-technical considerations. Sommerville's emphasis on avoiding premature technology decisions in early SDLC phases directly influenced the project approach: domain modelling, requirements elicitation, and workflow analysis were completed before committing to a specific technical stack. This ensured that initial outputs, such as UML diagrams and use cases, remained technology-agnostic while still aligning with TM354 best practices.

The transition to technology selection required a critical evaluation of cross-platform frameworks. Petzold's *Creating Mobile Apps with Xamarin.Forms* (2016) offered practical insight into mobile-native architecture. Still, they were ultimately deprioritised due to Xamarin's deprecation and the steep learning curve associated with adopting an unfamiliar tech stack within a constrained academic timeframe. Lawrence's *Introducing .NET MAUI* (2023) presented MAUI as a future-proof, cross-platform solution with advanced capabilities such as native camera integration;

however, organisational skillset limitations and maintenance considerations meant that MAUI was unsuitable for this prototype despite its technical merits.

React Native emerged as the most viable option, supported by Eisenman's *Learning React Native* (2019) and Boduch's *React and React Native* (2017), both of which provided implementation examples for structuring components, styling interfaces, and integrating mobile workflows used in the implementation phase of the project. These sources complemented TM352's coverage of mobile technologies, particularly the shared backend–frontend architecture pattern. Eisenman's practical demonstrations, combined with Boduch's broader focus on React ecosystem integration, informed the prototype's user interface navigation flows, styling conventions, and use of components such as *TouchableOpacity* to improve user interaction. Dongliang and Minjie's (2021) comparative study of cross-platform frameworks reinforced React Native's suitability for rapid proof of concept development, while also acknowledging its limitations for long-term enterprise deployment.

Database design decisions were guided by Elmasri and Navathe's *Fundamentals of Database Systems* (2021), which emphasised the importance of relational database models in ensuring data integrity, referential consistency, and auditability—requirements that are non-negotiable in compliance-driven environments such as maintenance inspections. NoSQL databases were rejected due to their weaker schema enforcement, and MySQL was ruled out for its lack of seamless integration with enterprise authentication and backup systems. Peres' *Modern Web Development with ASP.NET Core 3* (2020) offered additional insight into structured, layered application design, particularly in enterprise contexts.

To connect academic theory with practical implementation, Packt Publishing's *Master C# and SQL by Building Applications* (2022) was reviewed. Although it is focused on practitioners rather than peer-reviewed, this resource proved useful in enhancing understanding of three-tier desktop application architecture, which includes user interface, business logic, and data layers, in relation to the front-end, backend, and database layer within mobile applications. This layered model provided a conceptual framework for comparing desktop systems with hybrid mobile architectures. Specifically, the front-end React Native represents the user interface, the backend, such as Express.js or FastAPI, both of which are mentioned in TM352, handles business logic, and SQLite manages persistent storage. This comparative insight informed the design of the application as both a functional prototype and a blueprint for a future production-ready system.

Recognising the importance of offline-first capabilities, van der Lans' *The Definitive Guide to SQLite* (2022) was consulted. This material informed the approach to local data persistence identified during the analysis phase as critical for maintaining operational continuity in areas with unreliable Wi-Fi coverage.

Initially, organisational fit was a primary consideration due to internal IT policies favouring Microsoft-based platforms. However, after discussions with the technical

stakeholder, who previously oversaw the deployment of the organisation's internal systems and now acts as a technical advisor for this project, and further tutor guidance, it was agreed that the proposed solution should prioritise academic objectives. The chosen stack therefore reflects TM352 and TM354 learning outcomes while providing a structured, well-documented foundation for a later full-scale implementation in the organisation's preferred environment, outside the scope of TM470.

Beyond high-level architectural considerations, attention was also given to the design and implementation of the database layer, which underpins the application's integrity and auditability. While Elmasri and Navathe (2021) and Harrington (2016) provided the theoretical foundation for relational schema design, normalisation, and referential integrity, van der Lans' *The Definitive Guide to SQLite* (2022) was particularly relevant for the offline-first requirements identified in the problem domain. SQLite was selected due to its lightweight nature, ease of embedding within mobile applications, and ability to enforce constraints critical for compliance-driven environments.

To support practical implementation, additional practitioner-focused resources were also consulted. These additional resources included YouTube tutorials, online technical blogs, and developer-oriented texts. Although lacking peer-reviewed scholarship, these materials included step-by-step examples that clarified issues of integration, such as balancing primary and foreign keys, implementing user authentication, ensuring audit readiness, and connecting the database to a React Native user front end. Informal materials complement scholarly works by bridging, in this case, theory foundations and real-world work necessitated by rapid prototyping. Inclusion of them shows Sommerville's (2016) argument that software engineering practice is about, more often than not, combining good methods with pragmatic guidance to meet project constraints.

Resources, Planning, Risks and Lifecycle Review

1. Resources

Project success depended on the effective use of both technical and non-technical resources. These were categorised into software and hardware, knowledge and skills, and human interaction resources.

The resources were selected after evaluating alternatives and considering organisational policies, academic guidance, and the current skillset available.

Software and hardware

Initially, there was consideration of developing a Windows-based .NET system to support long-term organisational deployment, as discussed in Appendix 3.

However, due to the academic focus of TM470, the solution needed to be implemented in a way that built upon the existing knowledge from the TM352 Web, mobile, and cloud technologies module.

Ultimately, the final decision emphasised the prototype application best aligned with the academic requirements. This involved using technologies and approaches relevant to web and mobile development, along with selecting suitable frontend, backend, and API technologies. The aim was to create a realistic simulation of a cross-platform inspection system while enhancing key competencies such as mobile development, database integration, and object-oriented design.

In line with the TM354 Software Engineering module, modelling and prototyping activities were supported with a range of lightweight design tools. Tools such as Umlet, Lucidchart, and Visual Paradigm were used to create UML diagrams, including use cases, activity diagrams, sequence diagrams, and class models, providing a rigorous foundation for requirements validation.

While programs like Pencil UI were used for low-fidelity wireframes to illustrate system flow, they were also used to improve stakeholder engagement before any code was written.

For database exploration, Oracle SQL Modeller and SQLite Studio were employed to design and validate entity relationships before integration into the prototype.

During the prototyping phase, Visual Studio Code provided a flexible environment for iterative front-end development, supported by Node.js and Expo Go for rapid deployment on Android smartphones and tablets. This combination was chosen over Xamarin Forms, which, although considered (Petzhold, 2016), was deprioritised due to deprecation and steep learning requirements within the available timeframe (Dongliang and Minjie, 2021).

GitHub was used throughout for version control, ensuring traceability of iterations and providing evidence of a structured development process. This process can be reviewed either by inspecting the GitHub repository via the shareable hyperlink included in the Appendix [insert number] Version Control and Code Changes or by examining this supplementary material.

Knowledge and skills

Knowledge and skill resources were equally critical. Concepts from TM354 supported the adoption of structured SDLC (Software Development Life Cycle) phases. At the same time, transferable programming skills from the M250 (Object-oriented programming with Java) module studied as part of the level 2 syllabus facilitated the application of object-oriented design in both TypeScript and SQLite schema construction.

Practitioner sources such as Eisenman (2019), Boduch (2017), and van der Lans (2022) were blended with official framework documentation from Expo Go, SQLite, and

TypeScript to deepen practical competence. Additionally, a selection of targeted video tutorials available on platforms like YouTube, Pluralsight, and Stack Overflow reinforced technical understanding, where prior studies did not cover the content. This selection of sources provided the necessary balance between theory and practice to ensure that the prototype was both academically rigorous and technically functional.

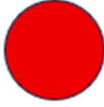
Human interaction

An essential factor in the project's progression was the strategic application of soft skills, with a particular emphasis on communication abilities. Over six years of professional experience collaborating with diverse stakeholders, I have significantly refined these skills. My proficiency in communication was further enhanced through presentations and discussions I facilitated during my tenure at this organisation. This experience allowed me to establish credibility, enabling me to effectively gather project requirements, even when stakeholder availability was limited due to my job transition.

Early user interface prototypes were demonstrated to engineers to validate usability, while discussions with a senior software engineer informed a bottom-up development strategy. Some of these interactions, also included in Appendix 3, reinforced feedback collection and iterative adaptation, ensuring user expectations remained central to the project.

2. Risk Assessment and mitigation

Anticipated risks were systematically recorded, graded for likelihood and impact, and visualised using a traffic-light system (Table 2.1). This approach mirrored professional practice, where risks are prioritised for monitoring and mitigation.

Risk	Likelihood	Impact	Rating	Mitigation
Limited stakeholder availability	High	High		Reliance on paper forms and workflow shadowing. Early prototypes were used to gather feedback before the implementation.
Time constraints	High	High		Structured timeline with buffer weeks. Prioritisation of core inspection functionalities.
Scope creep	Medium	High		Utilised MoSCoW prioritisation, with core features defined as “Must” and more advanced features categorised as “Should”, “Could”, or deferred or removed from the scope. Maintained a strictly scoped backlog, aligned with tutor advice. The scope was determined and agreed upon from the onset.
Lack of user requirements generation	Medium	High		Interviews with engineers, managers, and a technical advisor. Collection of the paper forms for data input.
Errors with data entry or retrieval.	Medium	High		Iterative schema testing in SQLite Studio or SQLite Editor. Database validation against conceptual diagrams.

Frontend and backend integration issues	High	Medium		Use of modular architecture. API endpoints were tested before binding with the user interface.
Limited exploration of alternative tools	High	Medium		Comparative evaluation of the pros and cons of using several different technologies. Justification of the final choice based on the project context, stakeholders' requirements and selected study modules.
Non-compliance with data security (GDPR)	Low	High		Limited data collection. Anonymised or mock-up records in prototype. Review of the GDPR guidelines (ICO, 2021).
Data integrity issues	Low	High		Application of relational constraints and database normalisation principles.
Lack of criteria for evaluating solutions	Low	Medium		Clearly defined metrics such as usability, integration fit, and performance. Early feasibility tests were conducted to determine the usefulness. Functional testing to assess usability and performance.

Debugging delays in unfamiliar frameworks	Medium	Medium		Adoption of Expo Go and reliance on official documentation. Incremental build-and-test cycles.
---	--------	--------	--	---

Table 3.2.1: Key project risks and mitigation strategies

3. Planning and progress

The project plan was organised around milestones based on the planning guidance from TM470. The planning framework adapted from the T227 module (Jones and Murray, 2014) was used to structure the project into clear, sequential phases: problem identification and definition, requirements gathering, design, implementation, testing, and closure. Each phase was linked to specific milestones and deliverables to aid progress tracking and risk management. This approach ensured that planning was methodical, traceable, and closely aligned with the SDLC principles recommended in the TM354 module.

The initial phases, including problem identification, domain modelling, and requirements elicitation, progressed largely on schedule. This success was supported by thorough early planning and the availability of existing paper forms and previous operational experience, with some evidence documented in the sections below.

Project phase	Milestones	Planned deliverables	Progress and adjustments
Planning & Investigation	Determination of project focus through analysis techniques	Planning documentation and confirmation of feasibility with stakeholders.	Completed successfully, however, initial emphasis on organisational fit delayed academic alignment until TMA02 feedback.
Requirements gathering	Collection of user requirements and analysis of existing inspection forms.	Defined set of user stories and instructions	Progressed as planned, paper forms substituted for limited stakeholder input. Risk mitigated by workflow shadowing and tutor guidance.
Low-level design	Identification of software and hardware requirements. Initial design of application structure.	High-level functional design. Draft of user interface wireframes; initial draft of user manual.	Delivered as planned, low-fidelity prototypes created in Pencil UI facilitated early feedback.

Detailed design	Design of application functionalities and interfaces. Database schema preparation.	Medium/high-fidelity screen layouts. Detailed class diagrams. Glossary and business rules.	Achieved, though entity refinement required iterative adjustments; database schema validated through normalisation.
Implementation	Development of prototype code in React Native with SQLite backend.	Prototype application capable of supporting login and at least one inspection workflow (Facilities and Machine Safety).	Implementation was delayed by integration challenges. Mitigation involved reallocation of 15% project hours to database/backend/frontend linkage.
Testing	Functional and usability testing of prototype features against specifications.	Documented test results. Confirmation that the system supports inspection recording and retrieval.	Iterative testing revealed asynchronous data handling issues, which were resolved using incremental schema validation and Expo Go deployment.
Release & closure	Delivery of proof-of-concept prototype and evaluation.	Prototype meeting academic goals, project documentation including reflections, UML artefacts, and final report submission.	Partially achieved. Some non-critical features (e.g., advanced reporting, predictive analysis) were deferred beyond the TM470 scope.

Table 3.3.1: Project phases, milestones, deliverables, and adjustments.

4. Project lifecycle selection

The lifecycle model combined sequential and iterative elements, reflecting both academic guidance and practical constraints.

Following the TM354 module materials and additional literature from Sommerville (2016), a Waterfall-style approach was applied to the early phases of requirements engineering, domain analysis, and system modelling.

This approach ensured that stable artefacts, such as UML diagrams, class models, and business rules, were produced based on the available domain documentation, such as maintenance inspection paper forms, checklists, and validated before development commenced.

Once prototyping began, an iterative strategy was adopted to accommodate the uncertainties of integrating new technologies and the reality of stakeholder feedback. An iterative approach allowed the prototype application to be developed incrementally, whilst being run on targeted physical devices, and refined based on performance and usability.

This hybrid approach balanced the strengths of Waterfall in providing structure with the adaptability of Agile principles, which are especially valuable in solo projects where scope must remain tightly controlled (Nandakumar, 2018).

The outcome was a proof-of-concept prototype that both extended TM352 and TM354 learning outcomes and offered a blueprint for a future production system. The lifecycle decision, therefore, was not merely a methodological preference but a pragmatic adaptation to academic aims and stakeholder expectations.

Project Work: Analysing and Solving a Practical Problem, demonstrating how your Computing and IT knowledge and skills have been extended

1. Domain modelling

Before developing the Maintenance Forms App prototype, hereafter called the system, it was essential to gain a thorough understanding of the operational environment in which it would be deployed. This stage focused on analysing the business domain to understand the problem and its context better.

Initial discussions with stakeholders supported the development of a problem statement, which the tutor later confirmed. Once the problem was defined, the domain modelling phase started. The aim was to provide a structured, technology-neutral perspective on the problem. By modelling the system independently of implementation details, it was possible to capture business processes, rules, and core classes, ensuring that the system's logic could be explained and refined without resorting to a premature solution. This structured approach reflects both standard practice in software engineering and the tutor's earlier feedback, which emphasised the importance of analysing the problem domain before progressing to solution design. Sommerville (2016) further stresses that separating the problem domain from solution design enables the creation of artefacts that remain valid regardless of whether the eventual delivery is mobile, desktop, or web.

An alternative would have been to prototype immediately and let the data structures "emerge". Although this second approach would bring fast implementation with quick feedback from engineers or the manager, it would risk a technical debt and a weak schema, which would not be suitable for a system that is meant to support compliance-heavy inspections (PUWER, 1998). Therefore, a more academic approach following software engineering principles laid out in the TM354 module materials was chosen for this project, involving the creation of class diagrams, system requirements and design structural models (The Open University, 2022) that would serve as a foundation for implementation.

To create the domain structural model with its conceptual classes, their potential attributes, and associations, further conversations with stakeholders were carried out, and existing paper forms were analysed to find common vocabulary, especially the domain nouns. Amongst the searched terms, a few have emerged: "Inspection", "Subcheck", "Site", "Zone", "Inspector", "Attachment", and "Result". These terms

formed the basis for building a glossary of terms, as presented in Table 4.1.1, with its initial phase extracted below. A full version of the glossary is available in Appendix 4.

Term	Meaning
Inspection	Routine check performed on factory equipment to verify it is operating safely and correctly. A single form for an item to be inspected.
Engineer	A user responsible for conducting inspections and submitting reports.
Manager	A user who reviews inspection results and generates reports.
Subcheck	Supplementary sub-check that the engineer must perform during an inspection of a single item. There could be multiple sub-checks assigned to a single inspected item.
Result	A pass/fail result for a single inspection.
Item	A piece of equipment or machinery is being inspected.
Site	A specific physical location that could have a company name, building number, address, etc.
Zone	A specific area of the physical location within a Site.
Attachment	Supplementary piece of evidence that can be attached as part of the inspection form, e.g. image or document

Table 4.1.1 Glossary of domain terms

The above glossary serves two primary purposes. First, it standardises vocabulary among engineers, managers, and the development team so that “zone”, “item”, or “result” have the same meaning to everyone. Second, it acts as a reference point for all subsequent artefacts and promotes consistency across diagrams, rules, and requirements. An alternative would have been to rely on informal, evolving terminology until late in the design, which could risk conflicting interpretations.

With key terms confirmed, the primary focus was on workflows. Routine inspections follow a consistent pattern, where engineers receive tasks, gather tools and the relevant paper inspection form, visit a location, carry out an inspection on an item and log the inspection results.

In contrast, managers then review the inspection forms, correct issues, and archive records for future audits. This process was depicted as an activity diagram (Figure 4.1.2 below) with two swim lanes (Engineer, Manager) to emphasise role interactions and sequence between the two users, serving as the foundation for business rules.

Sommerville (2016) highlights that business rules define the policies, constraints, and decision logic of a system, acting as a bridge between process modelling and system design. Based on this, a set of business rules was derived from the activity diagram to ensure that critical behaviours are consistently enforced by the system rather than relying on informal practice. These rules have been formalised and are presented in Table 4.1.3 below.

Rule ID	Rule description	Rationale	System Implication
BR1	Only one inspection can be active at a time.	Ensures traceability and prevents partial or duplicate records.	Database constraint to disallow multiple active inspections per engineer.
BR2	All subchecks must be explicitly marked (Pass/Fail/NA).	Prevents omissions from being misinterpreted as passes.	Validation in UI and database before submission.
BR3	Failed subchecks must include a comment and at least one attachment.	Provides evidence for audit and enables follow-up actions.	Conditional validation: submission blocked if evidence is missing.
BR4	An overall inspection result can only be "Pass" if all mandatory subchecks pass.	Reflects compliance requirements and ensures consistency.	Derived attribute logic in the backend that prevents direct editing.
BR5	Managers must review and approve an inspection before archival.	Guarantees oversight, improves data quality and the audit readiness.	Workflow rule: inspections flagged "pending approval" until validated.

Table 4.1.3 Business rules derived from the activity diagram

All the previous artefacts led to the successful creation of the list of potential classes and their attributes. The development of the conceptual classes was iterative, as shown in Appendix 5. Initially, inspections were modelled as two separate subclasses, *FacilitiesInspection* and *MachineSafetyInspection*, to reflect the paper-based separation of processes and preserve domain-specific terminology. Similarly, a single *Result* entity was introduced at the inspection level to reflect the way paper forms record an overall pass or fail. However, further analysis highlighted limitations in both designs. The inspection subclasses duplicated core attributes such as inspection date, inspector, site, zone, and item details. This duplication could introduce redundancy and inconsistency in later database design (Watt and Eng, 2014). Likewise, the inspection result could not guarantee compliance with business rules such as "failed items must include a comment and attachment" or "all subchecks must be explicitly marked". To address these issues, the model was refined into a single generalised *Inspection* class with an added *inspectionCategory* attribute, and *Result* entities were instead associated with individual *Subcheck* entities. This approach would reduce duplication, strengthen data integrity, and provide auditable evidence for each inspection step (Watt and Eng, 2014). The overall inspection result was then treated as a derived attribute calculated from

mandatory subcheck outcomes rather than one that was stored explicitly. Although these refinements introduced greater structural complexity, they ensured stronger compliance support, improved traceability, and closer fidelity to how engineers perform inspections in practice. This aligns with Sommerville’s (2016) guidance to embed business rules directly into the domain model rather than leaving them to informal practice.

This structured and analytical development of classes and their attributes led to a final polished version of the domain structural model that can be seen in Figure 4.1.4 below.

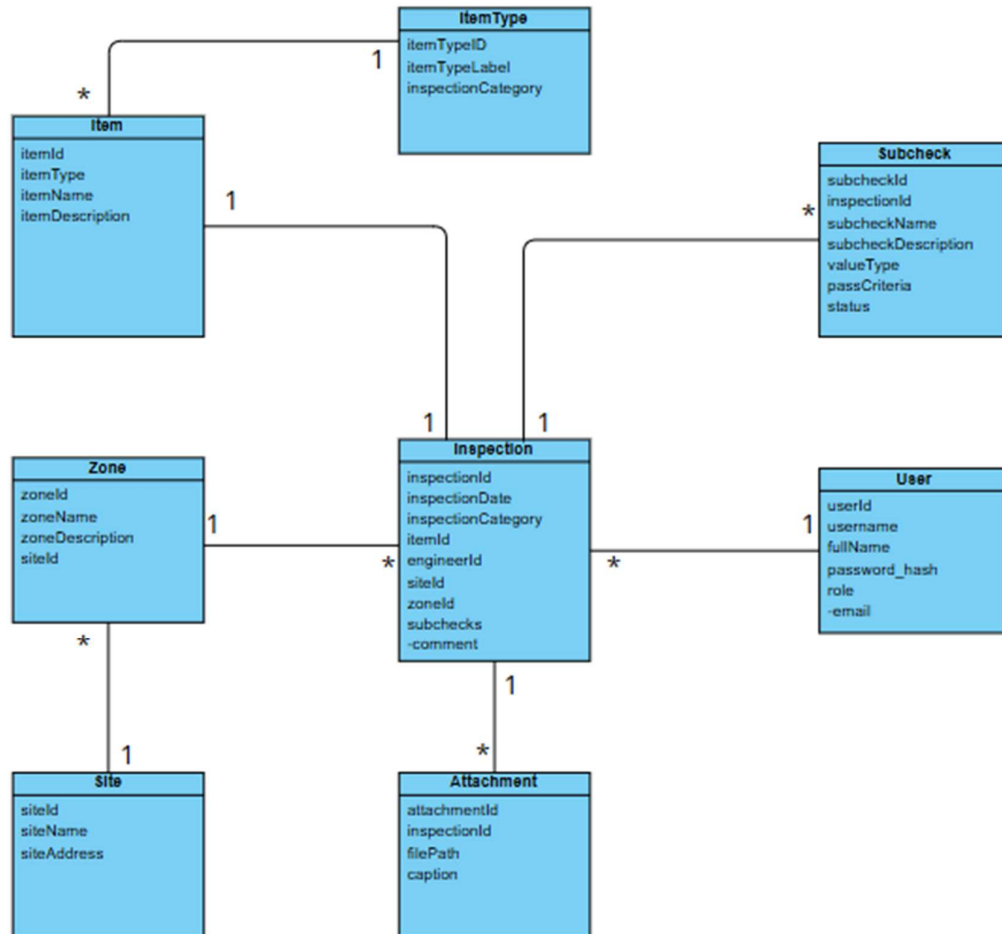


Figure 4.1.4 Domain structural model

Requirements

This section builds directly upon the insights gathered during the domain analysis and business process modelling stages described in the previous phase. The shift from conceptual analysis to system specification involved transforming the domain model into a coherent set of functional and non-functional requirements. These requirements define

how the system is expected to behave and the conditions under which it must operate. A goal-driven process, based on established practices such as those outlined by Sommerville (2016), guided this transition.

The requirements development process began with a combination of requirements elicitation techniques. Informal interviews with engineers and managers were conducted to gain a deeper understanding of their day-to-day roles, inspection processes, and to identify any social or organisational issues that could impact the system's use. In addition to these techniques, the personal experience gained from conducting inspection checks was also taken into account. These methods provided a solid foundation for the requirements document, which was later consulted with the tutor. Paper forms, collected during the domain modelling process and through informal conversations with engineers and managers, provided a clear understanding of user expectations and system usage. This research led to the identification of two main users and their activities: the Engineer, who performs inspections and submits results, and the Manager, who oversees inspections and may later handle reporting and follow-up tasks. These patterns of interaction were modelled as system actors and used to create use case diagrams that demonstrate how each interacts with the application. These diagrams are shown in Figures 4.2.1 and 4.2.2.

Although the potential for a System Administrator role was considered, it was deliberately excluded from this version of the system to focus on essential workflows and meet project deadlines.

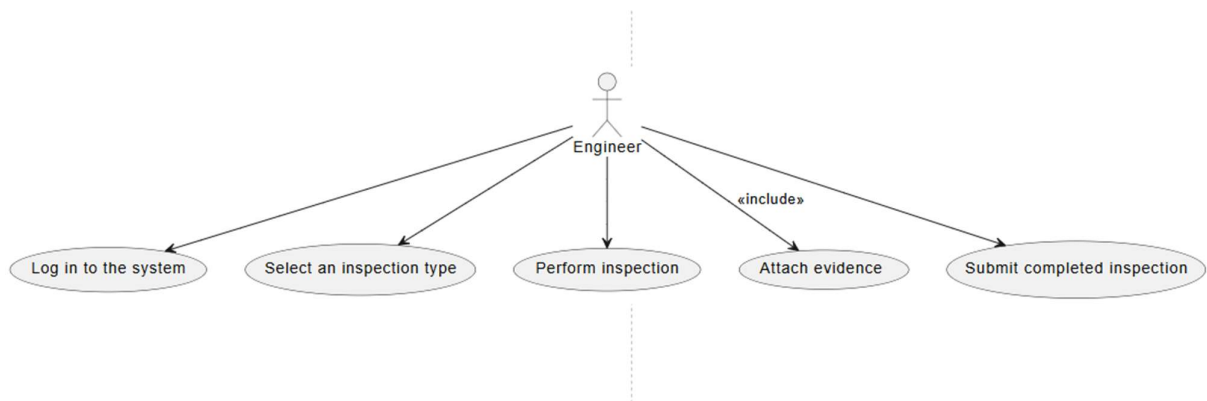


Figure 4.2.1 Use case diagram for an engineer completing the inspection

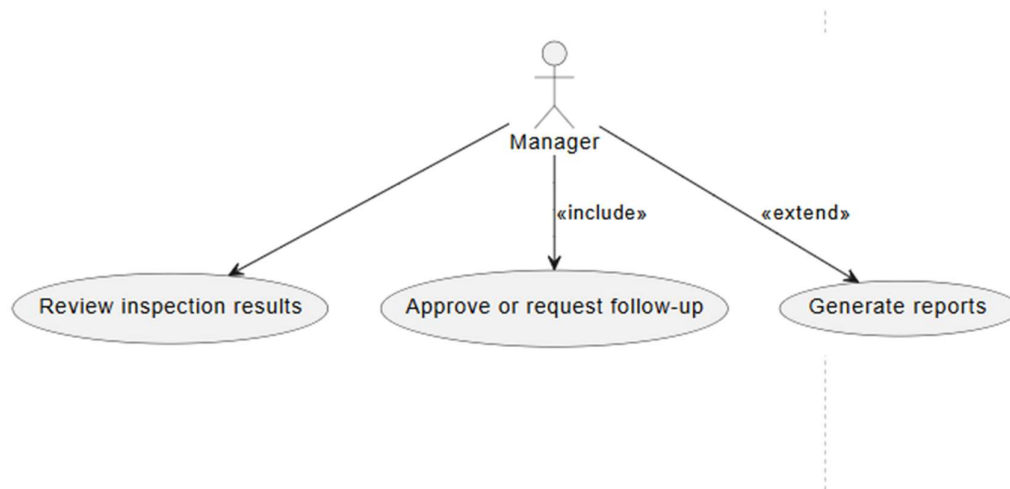


Figure 4.2.2 Use case diagram for a manager reviewing the inspection

The use case diagrams played an essential role in understanding user interactions and identifying the system’s expected behaviour. They also provided a framework for tracing functional requirements back to specific business processes, ensuring a user-focused system design. To capture these behaviours in more detail, structured use case descriptions were developed, including preconditions, postconditions, and alternate flows. These descriptions are included in Appendix 7 and form the basis of many functional requirements. An excerpt of two important elaborated use cases (Tables 4.2.3 and 4.2.4) below illustrates two significant user-system interactions for the engineers, who will be the primary users of the system it must support.

Use Case ID	UC-003
Actors	Engineer, Manager
Description	The user inserts their credentials and is being authorised to access the application.
Preconditions	The user inserts correct credentials
Basic Flow	1. The user inserts their correct login and password 2. System sends a request to the database to check the user credentials 3. System loads the application main screen 4. User is permitted to create the form, or use template
Alternative Flows	AF1: User cancels the login and receives the prompt that login process was cancelled AF2: System prevents login if the credentials are incorrect
Postconditions	User is authorised to access the application functionalities.

Table 4.2.3 Elaborated use case for user log-in functionality

<i>Use Case ID</i>	<i>UC-001</i>
<i>Actors</i>	Engineer
<i>Description</i>	The engineer completes a scheduled inspection by filling out digital forms, marking each sub-check as pass/fail, adding comments, and attaching photos where required.
<i>Preconditions</i>	The engineer is logged in and has been assigned at least one active inspection.
<i>Basic Flow</i>	1. Engineer selects the scheduled inspection from the list. 2. System loads the corresponding checklist (subchecks). 3. Engineer inspects each item and records a pass/fail result. 4. For failed items, the engineer adds comments and attaches at least one photo. 5. Engineer submits the inspection.
<i>Alternative Flows</i>	AF1: Engineer cancels the inspection before submission – system saves it as “In Progress.” AF2: System prevents submission if required evidence (comments/photos) is missing for failed checks (BR2).
<i>Postconditions</i>	Inspection status updated to “Submitted” and visible to the manager for review.

Figure 4.2.4 Elaborated use case for an engineer completing an inspection form

Functional requirements

To effectively manage scope and ensure the prototype's deliverability within project constraints, the MoSCoW prioritisation technique was adopted. This method enabled categorising requirements into Must, Should, Could, and Won't, which helped prioritise core functionality over future enhancements. This ordering framework offered clarity and focus during planning and implementation, helping to ensure the system remains both usable and fit for purpose.

The functional requirements were directly derived from detailed use cases and business rules, enabling the mapping of business rules to their corresponding functional requirements. For example, Business Rule 2 states that “failed items must include a comment and attachment,” which maps directly to Functional Requirement 4.5. Similarly, Business Rule 5, which states that “inspection is classed as pass if all mandatory subchecks for that inspection are pass”, underpins Functional Requirement 4.6.

Each functional requirement was prioritised using the MoSCoW technique, ensuring that only *Must Have* functionality, user authentication, inspection workflows, and evidence capture were implemented in the prototype.

Should and *Could* items (e.g., reporting, notifications) were included for completeness and to show forward planning, but were deferred to future iterations to keep the project deliverable within the available timeframe. The list below includes “Must” requirements only. For the full list of requirements, please see Appendix 8.

FR ID	Requirement	MoSCoW priority
FR 1.1	The system shall allow users (engineers, managers, admins) to log in using a username and password. (Must)	Must
FR 1.2	The system shall allow engineers to create an account with their full name, email, and password. (Must)	Must
FR 1.3	The system shall identify the logged-in user and display their name and role in the application header. (Must)	Must
FR 1.4	The system shall restrict functionality based on user roles (e.g. managers can review reports, engineers can only submit inspections)	Should

Table 4.2.5: FR1 - Functional requirements for user management

FR ID	Requirement	MoSCoW priority
FR 2.1	The system shall allow engineers to select or insert a site (e.g., “Unit 20”).	Must
FR 2.2	The system shall allow engineers to select or insert a location (e.g., “First Floor”, “Rear”) within the chosen site	Must
FR 2.3	The system shall ensure only zones belonging to the selected site are displayed.	Must

Table 4.2.6: FR2 - Functional requirements for site and location selection

FR ID	Requirement	MoSCoW priority
FR 3.1	The system shall allow engineers to select an inspection category: • Facility Check (e.g. Emergency Lights, Fire Doors) • Machine Safety Check (e.g. Die-Cut Machine, Print Machine)	Must
FR 3.2	The system shall dynamically load available inspection types based on the selected category.	Must

Table 4.2.6: FR3 – Functional requirements for inspection type selection

FR ID	Requirement	MoSCoW priority
FR 4.1	The system shall load the appropriate inspection form based on the selected inspection type.	Must
FR 4.2	The system shall display subchecks dynamically, based on machine type or facility type.	Must
FR 4.3	The system shall allow the engineer to record outcomes for each subcheck: • Boolean (Pass/Fail) (Must) • Numeric (e.g. readings, measurements) (Should)	Must
FR 4.4	The system shall allow the engineer to add an optional comment for each subcheck.	Must
FR 4.5	The system shall allow attaching optional evidence (Attachment class; as per the Business Rule 3)	Must
FR 4.6	The system shall automatically calculate the overall inspection result (Pass only if all mandatory sub checks pass).	Must
FR 4.7	The system shall allow the engineer to submit and save a completed inspection.	Must

Table 4.2.7: FR4 – Functional requirements for completing an inspection

Non-functional requirements

In parallel, a set of non-functional requirements was defined to ensure that the system not only works correctly but is also reliable, secure, and usable in the field. These included support for tablet-based interfaces, user authentication, offline functionality, and consistent UI design. Such features are essential in a workplace environment where mobile connectivity can vary, and tasks need to be completed efficiently.

NFR ID	Requirement	MoSCoW priority
NFR 1.1	The system shall provide a tablet-friendly interface with touchscreen-optimised controls.	Must
NFR 1.2	The system shall use clear labels and controls.	Must

Table 4.2.10: NFR1 – Non-functional requirements for usability

NFR ID	Requirement	MoSCoW priority
NFR 2.1	Forms shall load in under 3 seconds.	Should
NFR 2.2	Saving an inspection shall take no more than 2 seconds.	Should

Table 4.2.11: NFR2 – Non-functional requirements for performance

NFR ID	Requirement	MoSCoW priority
NFR 3.1	Only authenticated users shall access the system.	Must
NFR 3.2	Data shall be encrypted during transmission.	Should

Table 4.2.12: NFR3 – Non-functional requirements for security

As a result of this requirements phase, a complete and traceable set of specifications has been established, providing a solid foundation for the system’s design and implementation. Each requirement is informed by business needs, user input, and domain constraints, ensuring that the resulting system is both practical and aligned with its intended use.

Analysis

The analysis phase aimed to transition stakeholder concerns and problem narratives into formal representations that would later lead to design and implementation. This phase was heavily informed by structured requirements engineering and domain-driven design, emphasised by Sommerville (2016). By identifying key system components, including important data, user roles, and system functions, the analysis ensured the solution accurately reflected real-life maintenance inspections.

Key artefacts produced during this phase included a stakeholder-driven glossary, use case diagrams, conceptual class diagrams, and a set of prioritised functional and non-functional requirements. These artefacts not only provided a blueprint for future work but also acted as tools for early validation, allowing feedback loops with users before any code was written. This aligns with Sommerville’s principle of “specification as communication”(2016), where models help identify misunderstandings early.

Use case modelling was used to capture functional expectations from the perspectives of different user roles (engineer, supervisor). Each use case was broken down into sequences of interactions and alternative flows, some of which provided early design insights. For example, the “Perform Inspection” use case showed that engineers needed to operate in low-connectivity environments, directly shaping the decision to support offline features—a key non-functional requirement (NFR4.1).

The analysis also included domain modelling, where abstract concepts such as "Zone", "ItemType", and "SubCheck" were refined into a conceptual class diagram. This helped visualise relationships and hierarchies, aiding in both UI layout and database schema design. Unlike ad hoc data structures, this modelling ensured alignment between technical

implementation and user mental models consistent with TM354's (The Open University) recommendations for problem-space modelling.

Non-functional requirements (NFRs) were elicited through contextual enquiry and prioritised using the MoSCoW method. Requirements like security (NFR3.1) and usability (NFR2.1) were elevated due to their operational impact. This prioritisation directly influenced implementation choices, e.g., using password hashing and mobile-first design principles (N

Finally, trade-offs were recognised even at this early stage. While it was possible to over-specify inspection categories and item types, a more flexible solution was preferred, deferring some categorisation logic to the database layer. This decision reflects a **separation of concerns** between domain logic and user interface, a concept strongly reinforced in TM352 Block 2.

System modelling and design

Although the requirements provided a clear, abstract, and technology-agnostic view of the proposed system, further analysis was required to bridge the gap between high-level needs and a design suitable for implementation. Following Sommerville (2016), system modelling employed UML notations to capture and communicate system structure and behaviour across different viewpoints. This step was essential not only for documentation but to support traceability, design decision-making, and early validation, core theories of rigorous software engineering outlined in TM354.

The modelling process served three purposes: to identify system operations connected to use cases and functional requirements, develop conceptual domain classes into analysis classes with more detailed behaviour, and visualise temporal behaviour using sequence and state diagrams.

Sequence Diagram for Maintenance App Form Submission

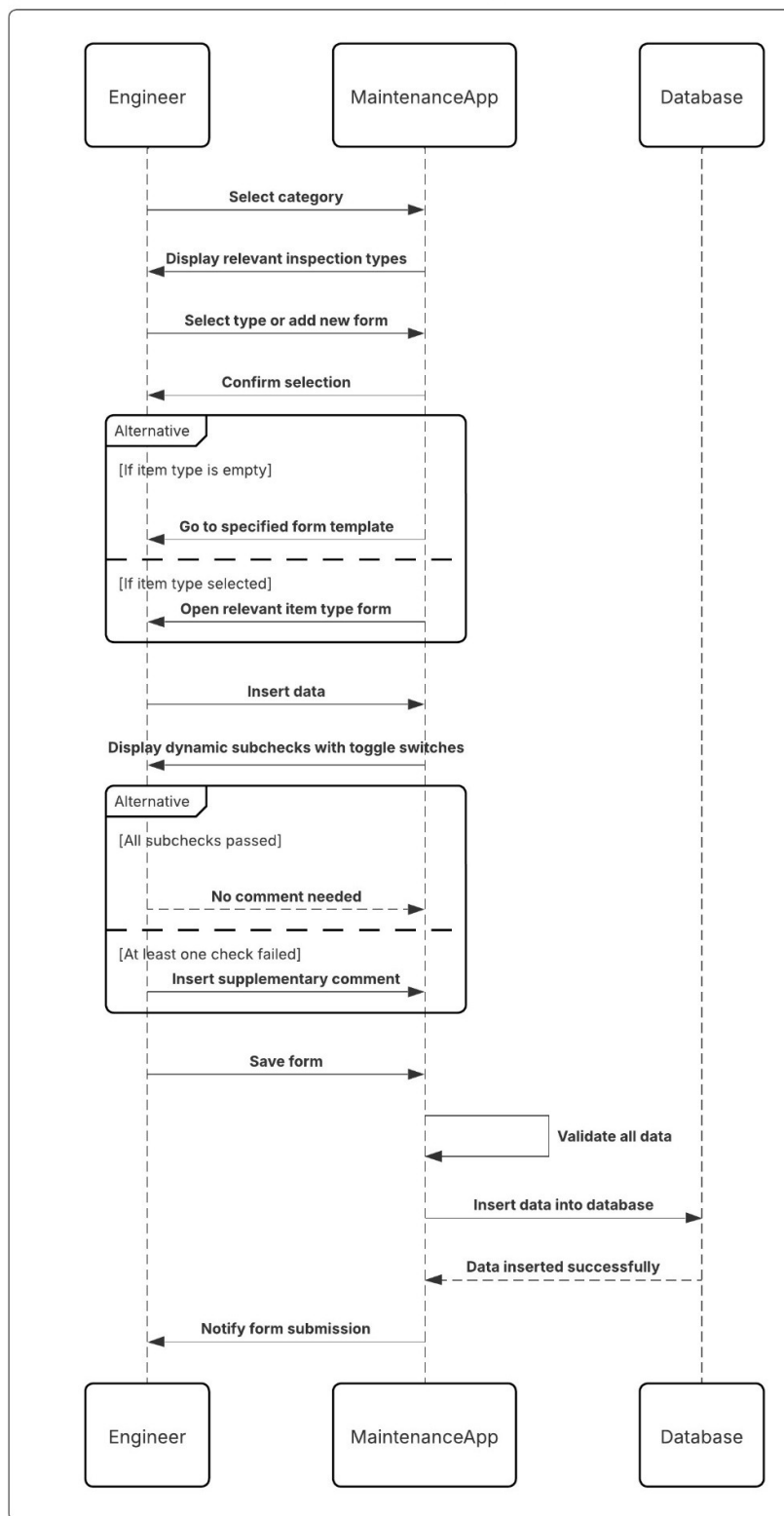


Figure 4.3.1 Sequence diagram for completing the inspection form

Sequence diagrams, like the one above (Figure 4.3.1), were particularly valuable in expressing dynamic system behaviour. They clarified how inspection data was fetched, submitted, and stored in response to user actions. These diagrams not only described the expected flow but also exposed gaps in logic, such as missing validation stages or inconsistencies in actor roles (Sommerville, 2016). These gaps were resolved by revisiting the use cases, demonstrating the iterative refinement typical in modern system development, emphasised by the author.

The formation of system operations, derived from functional requirements and refined through use case interactions, grounded the behavioural view of the system. These operations were then mapped to actors and requirements in a structured format (see Table 4.3.2). For example, the `submitInspection()` operation emerged from FR4.6 and involved interactions between the Engineer actor, the `InspectionForm` object, and the backend data store.

Use Case	System Operation	Input Parameters	Output / Effect
User logs in	<code>login(email, password)</code>	Email, password	Authenticated session or error
User registers	<code>registerUser(email, password, name)</code>	Email, password, full name	User record created
Site creation	<code>createSite(name, address)</code>	Site name, building number, site address	Site created or returned if <u>exists</u>
Zone creation	<code>createZone(siteId, name, description)</code>	Zone name, description, site ID	Zone created or returned if <u>exists</u>
Create inspection	<code>createInspection(data)</code>	Inspection form data	Inspection stored
Load item types	<code>getItemTypes(category)</code>	Category name	List of item types
Get subchecks	<code>getTemplatesByItemType(id)</code>	Item type ID or label	List of subchecks

Table 4.3.2 Primary system operations based on the use cases

Importantly, these operations formed the basis for both implementation logic (as API endpoints) and test cases demonstrating the value of early modelling in supporting testability and modularity. Operations not immediately necessary for the prototype, such as report generation and audit trails, were identified and prioritised for future development cycles using MoSCoW prioritisation.

In summary, system modelling and design in this project were not just simply a representational step, but an analytical process that shaped and informed downstream future coding activities. They enabled better stakeholder communication, surfaced design risks early, and strengthened the project's alignment with its functional goals.

Implementation

Wireframes and UI design

Following the requirements engineering and analysis stages, an early user interface implementation phase was initiated to validate the design logic and user interaction patterns before developing the underlying system logic. This approach aligns with Sommerville's (2016) recommendation to engage in iterative design and postpone high-cost architectural decisions until after stakeholder feedback has been collected. It also reflects TM354's emphasis on design as a progressive translation from abstract models to practical artefacts.

This phase did not involve finalised coding or full backend integration; instead, it prioritised usability validation through low-fidelity wireframes and clickable prototypes. These artefacts allowed for structured feedback, particularly from engineers who would be the system's primary users during routine inspections.

Initial designs were implemented using Figma and Pencil UI to create wireframes representing the system's core screens: login, main menu, inspection category/type selection, form completion, and result submission. These wireframes were directly traceable to use cases defined in Section 4.2, particularly FR1.1 (user login), FR3.2 (inspection type loading), and FR4.1–FR4.7 (inspection workflows). This mapping ensured that each visual artefact had a functional purpose and maintained consistency with the system model.

Special attention was given to the context of use. Engineers are expected to interact with the application via tablets during field inspections, often under time pressure or in low-connectivity environments. Consequently, the wireframe design followed mobile form interaction patterns (Neil, 2014), with minimal text input, large tappable controls, and screen layouts optimised for finger navigation—thus satisfying non-functional requirements NFR1.1 (tablet-friendliness) and NFR1.2 (clear controls).

Wireframes then evolved into clickable prototypes to simulate real-time user interaction flows. From login to selected site to inspection type choice and to completing an inspection This prototype was deployed to test devices using Expo Go, allowing engineers to evaluate the flow and screen clarity on mobile tablets. The simulation did not include backend logic but instead relied on placeholder interactions to reflect expected behaviour—consistent with the hybrid lifecycle model adopted in the project (see Section 3).

Feedback obtained from these early tests—such as requests for larger buttons, clearer user role display, and simplified navigation—was promptly incorporated. This prototyping cycle ensured that changes were made at a low technical cost, illustrating Sommerville’s (2016) argument that “requirements and design evolve together.”

Navigation was implemented using React Navigation (Eisenman 2019), specifically employing stack-based flows to replicate the actor-triggered operations defined during analysis. Contextual data, such as the engineer’s identity and inspection selection, was passed between screens using typed parameters—ensuring consistency, traceability, and separation of concerns across the UI flow. This method created a seamless bridge between conceptual understanding (from analysis) and interface logic, demonstrating effective application of model-driven engineering principles. It also enabled early alignment between future backend expectations and frontend structure, particularly for core operations like login (FR1.1), inspection setup (FR2.1–FR3.2), and result submission (FR4.7).

Database Technology Selection Rationale

In alignment with the project’s bottom-up implementation strategy (Section 4.5), SQLite was selected as the database management system instead of a traditional server-based SQL engine such as MySQL or PostgreSQL. This decision was informed by both technical suitability and contextual constraints surrounding mobile deployment. As highlighted in TM352 (The Open University, 2024b), embedded databases like SQLite offer significant advantages in mobile development scenarios where offline access, rapid setup, and minimal configuration are priorities. Given that engineers often perform inspections in remote locations with intermittent or no connectivity, SQLite’s file-based architecture allowed the system to operate independently of cloud services—directly supporting the planned non-functional requirement for offline capabilities (NFR4.1).

Additionally, the decision reflected the recommendations of the project’s technical stakeholder, who advocated a bottom-up development model beginning with the database. This approach, supported by Sommerville’s (2016) emphasis on managing architectural risk early, allowed foundational system components to be validated before integrating back-end logic or cloud APIs. SQLite’s (2025) seamless compatibility with React Native and Expo Go (Expo, 2025) further reduced technical overhead, enabling rapid iteration and schema evolution during the early stages of implementation (The Open University, 2024).

Moreover, from a security perspective, SQLite avoided the need to expose user data over a network during prototyping. Combined with secure password hashing using the bcrypt library (GitHub, 2025), this setup maintained a baseline level of data protection appropriate for early field testing. In summary, SQLite was chosen not as a compromise, but as a strategic enabler of fast, robust, and offline-capable development—meeting both functional

and non-functional requirements while supporting the project’s iterative lifecycle model (The Open University, 2024a; Sommerville, 2016).

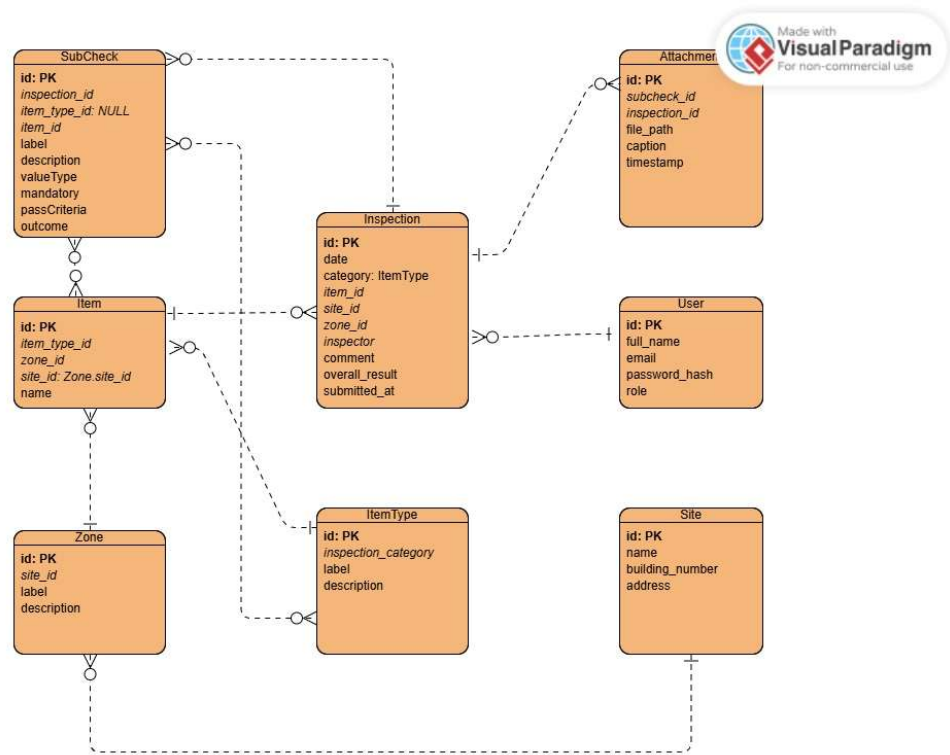


Figure 4.4.1 Database schema diagram

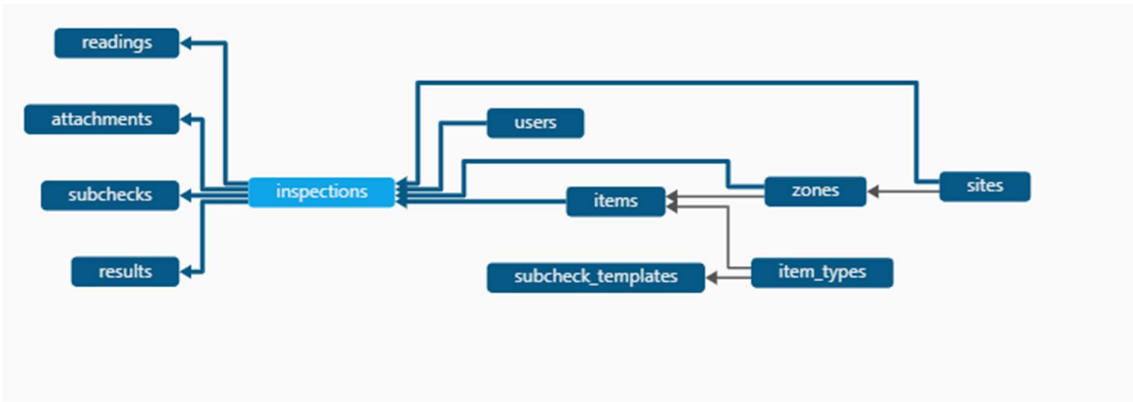


Figure 4.4.2 SQLite database schema diagram

For more information regarding implementation refer to github repository available at:
<https://github.com/LukaszCode/maintenance-forms-app-prototype/tree/main/maintenance-forms-app-prototype>

LSEPI and EDI Concerns

The development of the Maintenance Inspection Forms App necessitated a comprehensive analysis of various legal, social, ethical, and professional issues (LSEPIs) to ensure adherence to organisational policies, uphold professional standards, and align with broader ethical expectations. These considerations played a critical role in shaping both the research methodologies and the technical specifications of the project.

Legal considerations

This system is engineered to manage potentially sensitive inspection data, which includes attachment evidence linked to failed inspections. Although the app does not process personal employee data, it is imperative that the system remains compliant with the UK Data Protection Act of 2018 and the principles outlined in the General Data Protection Regulation (GDPR). This compliance is crucial to ensure that all inspection-related data is stored securely, accessed exclusively by authorised personnel, and retained only for as long as is necessary to fulfil compliance and audit requirements. The implementation of role-based access control, a vital feature identified during the requirements engineering phase, directly addresses these legal obligations. This approach restricts managerial capabilities to designated staff members (BR4), thereby minimising the risk of data breaches or unauthorised access.

During the research phase, all data gathered, ranging from interview notes to prototype feedback, was anonymised to protect participant identity. This data was securely stored on a password-protected device and is scheduled for permanent deletion following the completion of the TM470 assessment, as mandated by The Open University's Research Data Management policy, ensuring that we adhere to best practices in data stewardship.

Ethical concerns were meticulously addressed by ensuring that participation during the research phase was both voluntary and transparent. Engineers who provided feedback in the early stages of prototyping were given a detailed Research Study Participant Information

Sheet, in addition to signing a Consent Form (referenced as Appendix 1). These documents comprehensively outlined the study's objectives, methods for data storage, and the participants' right to withdraw from the study at any moment. Seeking formal consent underscores our professional obligation to maintain trust and respect for participant autonomy, which aligns with the guidelines set forth by the British Computer Society (BCS) in 2023.

Furthermore, the project took into account the ethical consequences of transitioning from a traditional paper-based workflow to a digital system. While the primary aim of this transition is to enhance operational efficiency, careful attention was given to the system design to avoid unnecessary surveillance of individual engineers. This decision was made to circumvent the creation of a work environment replete with excessive oversight, with a clear focus on prioritising equipment safety compliance over employee performance evaluation.

Professional considerations

The execution of this project adheres strictly to the BCS Code of Conduct, which emphasises the importance of acting in the public interest and exhibiting integrity in all professional undertakings. The decision to employ a bottom-up modelling approach—advocated by the organisation's senior software engineer—was grounded in professional rationale, as this methodology enhances data integrity and facilitates scalability for future development. Throughout the process, professional communication was prioritised; interactions with engineers and the IT manager were conducted with the utmost respect, demonstrating a commitment to explain technical concepts in a manner that was accessible and understandable.

Equality, diversity and inclusion

The design of the system is centred on accessibility and inclusivity. Key interface elements, such as large, touch-friendly buttons and clearly labelled options, were emphasised based on initial feedback from engineers. Special consideration was given to practical challenges posed by working in environments that may be poorly lit or physically constrained. The system's user interface is intentionally straightforward, avoiding assumptions about users' pre-existing technical knowledge, thereby ensuring that navigation remains intuitive for all users.

Participation in research activities was open to all engineers and managers, irrespective of their role or level of seniority, reinforcing our commitment to inclusivity. Careful scheduling of discussions was undertaken to accommodate the availability of all

participants, thereby respecting their work patterns and minimising disruptions to their schedules.

Personal Development, Review and Reflection

What were the challenges?

During the course of this assignment, I encountered several challenges that ultimately impacted my performance. One significant issue was my underestimation of the time required to complete the report portion of the assignment. Initially, I believed I could balance both the development of the application and the writing of the report simultaneously, but as the deadline approached, it became clear that I had miscalculated the time needed for thorough research and proper presentation of my findings.

Moreover, I found that my perfectionist tendencies significantly affected my approach to the project. Rather than focusing on creating a streamlined and fully functional application that incorporated the essential frontend, backend, and API components necessary to satisfy the key objectives I had outlined in my requirements, I became overly absorbed in adding extra functionalities. My goal was to make the application stand out, which inadvertently consumed more time and resources than I had originally planned. This decision ultimately left me scrambling to produce a robust report, which fuelled a sense of frustration and stress as I struggled to meet the assignment's requirements.

Another early mistake I made was my eagerness to produce a business solution too quickly. I mistakenly prioritised finding a solution to a real-world problem over understanding the project's broader goals. This rush led to confusion about the assignment's objectives and resulted in the submission of several poor-quality assignments, especially when compared to the standards I had set in my other modules. The root of this confusion was my failure to thoroughly review the module guides. I became overly focused on generating a solution rather than adhering to a structured approach aimed at producing tangible artefacts for the project. This approach was detrimental, leading to low grades on two of my tutor-marked assignments and forcing me to expend significant effort to improve my performance on the third assignment.

Alongside these challenges, I was navigating a demanding personal situation. While I was studying TM470, I was also enrolled in TM352, which required significant time and effort.

My experience with web and mobile development was limited to the TT284 module I completed in my second year and a few small tutorial-led projects I pursued as a hobby.

Additionally, my family life underwent several changes, including my daughter starting secondary school and my transition to a new job in a different industry. These changes disrupted my routine and made it difficult to access support. Moreover, for a significant portion of the module, I found it challenging to have personal one-on-one discussions with my tutor. It wasn't until late April, after changing jobs, that I moved from a permanent late shift to a more flexible shift pattern. This new schedule consisted of alternating weeks of early morning and afternoon shifts, which complicated my ability to arrange meetings with my tutor during their available hours.

What went well?

Despite the challenges I faced, I did experience some positive moments, particularly during the implementation stage of the project. I took great pleasure in engaging with various resources, including books, online articles, and YouTube tutorial videos, all of which provided valuable insights into software development. Furthermore, I had the opportunity to brainstorm ideas with my technical advisor, a retired software engineer with extensive experience in building various systems. His feedback was invaluable; he offered advice on the best strategies for approaching the project and implementing the application effectively.

However, he was unaware that the focus of this module was less about producing software products and more about analysis and project management. This structure of the module reminded me of the T227 course, "Change, Strategies and Projects at Work," which I studied in my second year. I recognised certain similarities in the planning and resourcing components of both modules, which helped me understand the importance of conducting thorough user requirement elicitation. As a result, I provided user stories that are now included in the appendices of my submission.

Reflections and what I would do differently

Reflecting on my journey through this module, I have gained several key insights that will inform my future work. I have learned the importance of proper time management and the need to approach projects with a structured plan that prioritises not just development, but also comprehensive analysis. My experiences have shown me that quality over quantity should be my guiding principle—focusing on completing a solid solution rather than adding unnecessary features will streamline my workflow.

If I were to undertake this project again, I would prioritise allocating time specifically for the report and ensure that I engage with all module resources thoroughly from the outset. I would also make it a point to maintain open lines of communication with my tutor and seek feedback more consistently, rather than waiting until later in the module when my schedule allows.

In summary, this assignment has been a profound learning experience, marked by both challenges and successes. I am now better equipped with knowledge and strategies that will serve me well in future endeavours, and I look forward to applying these lessons as I progress further in my studies.

References

- Alleva S. (2024). Mitigating the Risk of Misinterpretation, *Balancing Visuals and Text in Documentation Writing*, Paligo, Article [online] Available at: <https://paligo.net/in-depth/balancing-visuals-and-text-in-technical-documentation/> (Accessed 17 March 2025)
- BCS (2023) *Code of Conduct*, British Computer Society. Available at: <https://www.bcs.org/membership-and-registrations/become-a-member/bcs-code-of-conduct/> (Accessed 05 April 2025)
- BSC (2023) *Prioritising sustainability over profit*, British Computing Society, Article [online], Available at: <https://www.bcs.org/articles-opinion-and-research/prioritising-sustainability-over-profit/#:~:text=Extensive%20research%20has%20shown%20a,their%20peers%20in%20multiple%20areas>. (Accessed 15 May 2025)
- Boduch, A., Derks, R. and Sakhniuk, M. (2022) *React and React Native: Build cross-platform JavaScript applications with native power for the web, desktop, and mobile*. Fourth edition. Birmingham: Packt Publishing. Available at: https://library-search.open.ac.uk/permalink/44OPN_INST/j6vapu/cdi_askewsholts_vlebooks_9781803249995 (Accessed 06 June 2025)
- Conroy, L. *et al.* (2023) 'Medical physics practice guideline 4.b: Development, implementation, use and maintenance of safety checklists', *Journal of applied clinical medical physics*, 24(3), pp. e13895-n/a. Available at: <https://doi.org/10.1002/acm2.13895>. (Accessed 13 March 2025)
- Coronel, C. and Morris, S. (2018) *Database Systems: Design, Implementation, and Management*. 13th edn. Boston: Cengage Learning. Available at: <https://unidel.edu.ng/focelibrary/books/Database%20Systems%20%20Design,%20Impleme>

[ntation,%20%20Management%20by%20Carlos%20Coronel,%20Steven%20Morris%20\(z-lib.org\).pdf](#) (Accessed 30 May 2025)

Eisenman, B. (2016) *Learning React Native: Building Native Mobile Apps with JavaScript*. 1st edn. Sebastopol: O'Reilly Media, Incorporated. Available at: https://library-search.open.ac.uk/permalink/44OPN_INST/j6vapu/cdi_askewsholts_vlebooks_9781491929070 (Accessed 29 May 2025)

Elmasri, R. and Navathe, S. (2021) *Fundamentals of Database Systems*. 7th edn. Harlow: Pearson Education. Available at: <https://www-sciencedirect-com.libezproxy.open.ac.uk/science/article/pii/S0026269297809603> (Accessed 02 June 2025)

Esposito, D. (2020) *Modern Web Development with ASP.NET Core 3*. 2nd edn. Birmingham: Packt Publishing. Available at: https://library-search.open.ac.uk/permalink/44OPN_INST/j6vapu/cdi_walterdegruyter_books_10_0000_9781789614282_001 (Accessed 02 June 2025)

Fortra (2025), *What is Role-Based Access Control (RBAC)? Examples, Benefits, and More*, Digital Guardian Data Protection Blog, Accessible at: <https://www.digitalguardian.com/blog/what-role-based-access-control-rbac-examples-benefits-and-more> (Accessed 18 Feb 2025)

Glynn L. (2023), *Benefits of Changing from Paper to Digital Forms with Mini Case Studies*, forms2, Blog Post, 21st Sept 2023, Available at: <https://forms2mobile.com/blog/benefits-of-changing-from-paper-to-digital-forms-mini-case-studies/> (Accessed 14 May 2025)

GWP (2024), *Minimising environmental impact*, Environment, Sustainable packaging company, GWP Group, Available at: <https://www.gwp.co.uk/sustainable-packaging/environment/> (Accessed 14 May 2025)

HSE (1998), *Inspection of work equipment*, PUWER, Health and Safety Executive, Open Government Licence, Available at: <https://www.hse.gov.uk/work-equipment-machinery/inspection.htm> (Accessed 18 Feb 2025)

HSE (1998), *Inspection of work equipment*, PUWER, Health and Safety Executive, Open Government Licence, Available at: <https://www.hse.gov.uk/work-equipment-machinery/inspection.htm> (Accessed 18 Feb 2025)

HSE (1998), Health and Safety basics for your business, *Have the proper workplace facilities*, Health and Safety Executive, Open Government Licence, Available at: <https://www.hse.gov.uk/simple-health-safety/workplace-facilities/index.htm> (Accessed 21 Feb 2025)

ICO (2021) *Guide to the UK GDPR*, Data protection by design and default, Information Commissioner's Office Available at: <https://ico.org.uk/media/for-organisations/guide-to-data-protection/guide-to-the-general-data-protection-regulation-gdpr-1-1.pdf> (Accessed 06 April 2025)

Neil, T. et al. (2014) *Mobile design pattern gallery : UI patterns for smartphone apps*. Second edition. Edited by M. Treseler. Sebastopol, California: O'Reilly. Available at: https://library-search.open.ac.uk/permalink/44OPN_INST/la9sg5/alma9952608209602316 (Accessed 02 June 2025)

Nandakumar, M. (2018) *Minimum Viable Product*, Chapter 6: Managing the Scope of an Impact Driven Product, Lean product management: successful products from fuzzy business ideas, 1st ed. Birmingham: Packt. Available at: https://library-search.open.ac.uk/permalink/44OPN_INST/la9sg5/alma9952714812802316 (Accessed 06 Jun 2025)

Open University (2025), *Evaluation using PROMPT*, Finding Resources for your assignment, Help and Support, Available at: <https://www5.open.ac.uk/library/help-and-support/advanced-evaluation-using-prompt> (Accessed 01 Mar 2025)

Office for National Statistics (ONS), (2024), UK Average Earnings by Role and Industry.

Paul, A. and Nalwaya, A. (2019) *React Native for Mobile Development : Harness the Power of React Native to Create Stunning iOS and Android Applications*. 2nd ed. 2019. Berkeley, CA: Apress. Available at: <https://doi.org/10.1007/978-1-4842-4454-8>. (Accessed 05 Mar 2025)

SFG20 (2024), *Ultimate Guide To Compliance In Facilities Management*, Liam Sandford, 27th of June 2024, SFG20 [online], Available at: <https://www.sfg20.co.uk/blog/facilities-management-compliance-ultimate-guide> (Accessed 13 March 2025)

Simone A. (2020), *Flutter Projects*. Packt Publishing. Available at: https://library-search.open.ac.uk/permalink/44OPN_INST/j6vapu/cdi_safari_books_v2_9781838647773 (Accessed 03 Mar 2025)

Dongliang Y., Minjie H. (2021). *A Comparative Study of Cross-platform Mobile Application Development*, Available at: https://www.researchgate.net/publication/357898491_A_Comparative_Study_of_Cross-platform_Mobile_Application_Development (Accessed 15 April 2025)

Sommerville I., (2016) *Software Engineering*. 10th edition Harlow: Pearson Education, Available at: <https://dn790001.ca.archive.org/0/items/bme-vik-konyvek/Software%20Engineering%20-%20Ian%20Sommerville.pdf> (Accessed 26 May 2025)

Windmill, E. (2020) *Exploring Cross-Platform Mobile Development with Flutter, React Native, and Xamarin*. Available at: <https://www.manning.com/books/exploring-cross-platform-development> (Accessed 02 Mar 2025)

IEEE (2022), H. Zhu, H. Ebel, D. Scheinert, F. Schmidt, J. Altenkirch and O. Kao, *Scalable and Data-driven Decision Support in the Maintenance, Repair, and Overhaul Process*, 2022 IEEE International Conference on Industrial Engineering and Engineering Management (IEEM), Kuala Lumpur, Malaysia, 2022, pp. 0344-0348, Available at: <https://ieeexplore.ieee.org/document/9989791> (Accessed 18 Feb 2025)

Jones R, Murray N. (2014), Project planning, Gantt chart, p. 142-144, Change, Strategy and Projects at Work, The Open University, Routledge

Workflowy (2025), *Moscow*, Guides, Available at: <https://workflowy.com/systems/moscow-method/> (Accessed 16 April 2025)

The Open University (2024) TM354 Software Engineering. Milton Keynes: The Open University. (Accessed 13 May 2025)

The Open University (2024) TM352 Web, Mobile and Cloud Technologies. Milton Keynes: The Open University. (Accessed 10 June 2025)

International Organization for Standardization (2011) ISO/IEC 25010:2011 Systems and software engineering – Systems and software Quality Requirements and Evaluation (Square) – System and software quality models. Geneva: ISO. (Accessed 08 May 2025)

GitHub (n.d.) *bcrypt.js – A bcrypt library for Node.js*. Available at: <https://github.com/dcodeIO/bcrypt.js> (Accessed: 24 Jun 2025).

Bibliography

Epilogue

Appendix 1 – LSDEPI Forms



TMA02 LSEPI Form

I confirm that I have read and understood the resources on the TM470 module website on working with human participants. |

[Legal, Social, Ethical and Professional issues: 3 Working with human participants | OU online](#)

[Legal, Social, Ethical and Professional issues: Appendix A Guidelines for conducting research with human participants | OU online](#)

I understand that The Open University is unable to offer liability insurance to cover any negative consequences of my actions in carrying out research as part of this module.

Name: Lukasz Brzozowski

Date: 08/05/2025



EMA LSEPI Form

I confirm that I have read and understood the resources on the TM470 module website on working with human participants;

[Legal, Social, Ethical and Professional issues: 3 Working with human participants | OU online](#)

[Legal, Social, Ethical and Professional issues: Appendix A Guidelines for conducting research with human participants | OU online](#)

I confirm that all participants in my project were given a copy of my Participant Information Sheet which I have included in Appendix 1.

I confirm that all participants in my project were given a copy of my Participant Consent Form, a blank copy of which I have included in Appendix 1.

I confirm that I have collected signed Participant Consent Forms from all of the participants in my project.

I understand that The Open University is unable to offer liability insurance to cover any negative consequences of my actions in carrying out research as part of this module.

Name: Lukasz Brzozowski

Date: 15/07/2025

RESEARCH STUDY PARTICIPANT INFORMATION SHEET

Study Title:

*Exploring the Digitalisation of Paper-Based Maintenance Inspections
in a Factory Environment*

Researcher:

Łukasz Brzozowski

Final-year Computing and IT (Computer Science) student
The Open University
Email: lukabrzo@gmail.com

Alternative Contact (Independent):

Pat Ryser-Welch TM470 Module Tutor
Email: pat.ryser-welch@open.ac.uk

Invitation paragraph

You are being invited to take part in a research study. Before you decide whether or not to take part, it is important for you to understand why the research is being carried out and what it will involve. Please take time to read this information carefully.

You have been selected because of your experience in performing or reviewing monthly maintenance inspections within the organisation. Your input will help ensure that the proposed digital system reflects the actual needs of its users. Choosing to participate or not participate will have **no impact** on your role, evaluation, or progression at work.

Research Purpose

The purpose of this study is to explore how routine paper-based maintenance inspections can be improved by introducing a digital system. The research will focus on understanding the information captured in paper forms, how inspections are carried out, and how the proposed digital solution could improve efficiency, accuracy, and compliance.

General information about the research study and collected research data

Include the following information here, in terms that a lay person can easily understand:

The purpose of the research.

Type of research intervention, e.g. interview, survey, observation, etc.

How long the study will run.

What will I be asked to do if I agree to take part?

If you agree to take part, you may be asked to:

- Participate in **informal discussions or short interviews** (10–20 minutes) about how you currently complete or review inspections.
- Review early **wireframes and interactive user interface prototypes** and provide feedback on usability and workflow.

Participation is entirely **voluntary**. If you agree to take part, you will be asked to sign a consent form to confirm your understanding.

Benefits:

Your feedback will help ensure the system meets real operational requirements, potentially improving efficiency and reducing administrative workload.

Risks/Disadvantages:

There are no expected risks or disadvantages. Discussions will focus only on existing workflows, not on individual performance.

How will the data I provide be used?

Data storage: Notes and feedback will be stored securely on a password-protected personal computer. No paper copies will be kept.

Confidentiality: All responses will be anonymised. Your name will not appear in any report. Feedback will be reported as generalised observations (e.g., “engineers preferred larger buttons for use in poorly lit areas”).

Retention: Data will be stored only until TM470 assessment is complete and will be deleted by **30 November 2025**.

Sharing: Only anonymised findings will be included in the final project report, which will be submitted to The Open University for assessment. No information will be shared with third parties.

Your right to withdraw from the study

- You can withdraw from the study at any time by informing me verbally or via email.
- You can request for your data to be deleted up until **30 October 2025**, after which the data will have been anonymised and aggregated for analysis.

How do I agree to take part?

If you are happy to participate, you will be asked to **sign a consent form** (Appendix 1.1) before any data is collected.

Thank you

Thank you for taking the time to read this information sheet. Your contribution is highly valued and will play an important role in improving the system design.



INFORMED CONSENT FOR DIGITAL MAINTENANCE INSPECTION SYSTEM STUDY

This template can be used by researchers to gain informed consent when collecting data from people using questionnaires, observations, interviews, diaries, focus groups, video recordings, etc. It pays particular attention to how data can be curated and made

available for future use, and also addresses all the standard requirements of a consent form.

It is consistent with the requirements of the General Data Protection Regulation (GDPR).

The template should be adapted to the requirements of your particular study using the notes and suggestions provided.

1. Black text forms the standard content of a consent form
2. [Insert specific information in the highlighted square brackets]
3. Text notes in the grey boxes provide guidance only and are to be removed in the final consent form
4. Blue text indicates optional statements to add

INFORMED CONSENT FOR Developing a Digital Maintenance Inspection System

Researcher: Łukasz Brzozowski, TM470 student, The Open University

Contact: lukabrzo@gmail.com

Independent contact: Pat Ryser-Welch TM470 Tutor, The Open University, **pat.ryser-welch@open.ac.uk**

Please highlight your choice by clicking inside the appropriate box.

I have read and understood the information sheet for the following study: “Developing a Digital Maintenance Inspection System” or it has been read to me. I have been able to ask questions about my participation and my questions have been answered to my satisfaction.	YES ☒	NO ☐
I consent voluntarily to be a participant in this study and understand that I can refuse to answer questions I am not comfortable with and I can withdraw from the study at any time by contacting Łukasz Brzozowski, email: lukabrzo@gmail.com up until 30 September 2025 without having to give a reason.	YES ☒	NO ☐
I understand that taking part in the study involves providing feedback on wireframes, clickable prototypes, and user interface designs via informal interviews and discussions. Describe in a few words how information is captured, using the same terms as you used in the participant information sheet, for example: an audio-recorded interview, a video-recorded focus group, a survey questionnaire completed via a web browser, etc	YES ☒	NO ☐
Depending on the study, it may be appropriate to list some additional consent items here for emphasis, e.g: I agree to photos being taken of paper forms used for analysis (but no identifiable personal data will be collected). I understand that taking part in the study has no risks beyond ordinary workplace discussions.	YES ☒ YES ☒	NO ☐ NO ☐

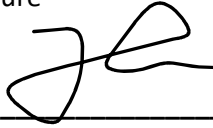
2. Use of the information in the study

I understand that the information I provide will be used for my TM470 academic project report and may be summarised in anonymised form in the final dissertation submitted to The Open University. List the planned outputs, e.g. reports, publications, website, video channel etc., using the same terms as you used in the participant information sheet. For many projects, it is appropriate to state that information will only be shared in fully anonymised form	YES ☒	NO ☐
I understand that personal information collected about me that can identify me (e.g., my name or role) will not be shared beyond the study team.	YES ☒	NO ☐
I understand that my data will be stored on a password-protected computer and deleted after my final EMA submission (September 2025).	YES ☒	NO ☐
Depending on the study, it may be appropriate to list some additional consent items here for emphasis, e.g: I agree to being quoted anonymously in the final report.	YES ☒	NO ☐

3. Future use and reuse of the information by others

I give permission for anonymised data (e.g., generalised feedback summaries) to be used in future academic discussions or as supporting evidence for postgraduate applications, excluding commercial use. • Specify in which form the data will be deposited, e.g. de-identified (anonymised) transcripts, audio recording, survey database, etc.; and if needed repeat the statement for each form of data you plan to deposit. (Fully anonymised data will be appropriate in most cases) • Specify whether deposited data will be de-identified (anonymised), and how. Make sure to describe in detail in the participant information sheet. • Specify whether use or access restrictions will apply to the data in future, e.g. exclude commercial use, apply safeguarded access, etc.; and discuss these restrictions with the repository in advance • Destruction of consent forms – consent forms should be kept for as long as the research data are retained (by the researcher or an archive). The original consent forms can be digitised and stored securely (encrypted), permitting the originals to then be destroyed securely by means of shredding. GDPR – How does this affect RDM (Research Data Management) is a very useful document.	YES ☒	NO ☐
You may wish to offer participants the opportunity to receive a summary of the findings of the study. I would like to receive a copy of the summary of the findings of this study. <i>Please insert your email address in the space below if you answer 'yes'</i> Email address	YES ☐ YES ☐ YES ☐	NO ☒ NO ☒ NO ☒

4. Signatures

Name of participant [in CAPITALS] _____Joe Lane_____	Signature  _____ (electronic signatures may be accepted)	Date _____15/06/2025_____
--	--	----------------------------------

For participants unable to sign their name, please mark the box instead of signing ☐

Please include the statement below at the bottom of the consent form.

AS your project is not undergoing formal HREC review:

This research project conforms to and complies with the OU Human Research Ethics Committee's conditions for exemption from formal review.

Placeholder

Appendix 2 – Project Log Extracts

Date	Activity	Comments	Time Spent (hrs)
04-03-2025	Initial meeting with manager, engineers, senior software developer, IT manager	Kickoff and scope-setting meeting; discussed operational challenges, existing paper process, and digitalisation goals.	3
05-03-2025	Communicating with tutor and discussing proposal	First formal check-in to validate project direction and confirm alignment with TM470 learning outcomes.	1
06-03-2025	Collecting paper-based inspection forms	Retrieved current inspection forms (emergency lights, fire doors, machine safety checks) from factory floor.	2
07-03-2025	Analysing layout and structure of paper forms	Reviewed formatting and extracted field names from forms; created initial notes for data mapping.	3
08-03-2025	Identifying data captured in current forms	Listed recurring data fields (e.g., site, zone, inspector name, pass/fail outcomes).	2

09-03-2025	Domain modelling and system context diagrams	Drafted initial conceptual models based on extracted fields; included main entities (User, Inspection, Result).	4
10-03-2025	Mapping paper form fields to domain model	Mapped real-world forms to model attributes; added preliminary multiplicities based on business rules.	3
11-03-2025	Requirements gathering and problem analysis	Solo analysis of workflows based on operational experience; started defining preliminary business rules.	4
12-03 to 14-03-2025	Researching technologies (C#, SQL Server, Xamarin.Forms, NoSQL)	Investigated tech stack; considered organisational requirements and early feasibility of cross-platform tools.	8 (over 3 days)
15-03 to 17-03-2025	Researching relational vs non-relational approaches	Compared SQL Server to NoSQL for data consistency and audit needs; decided relational is more suitable.	6 (over 3 days)
18-03-2025	Creating ER diagrams and refining models	Drafted ERD for SQL database; validated with senior software engineer.	3
19-03-2025	Designing SQL Server schema	Converted ERD into table schemas and primary/foreign keys.	3
20-03-2025	Discussion with senior software engineer	Reviewed DB design; recommended bottom-up approach starting with data layer.	1
21-03-2025	Tutor consultation	Tutor advised to prioritise TM470 academic outcomes before full-scale system development.	1
22-03 to 30-03-2025	Domain modelling refinement and business rules	Validated business rules (e.g., mandatory attachments for failed checks) with engineers; revised conceptual model.	12 (over 5 sessions)
01-04 to 04-04-2025	Creating wireframes and UI mockups	Designed clickable wireframes in Figma; shared with engineers for usability feedback.	8
05-04-2025	Feedback session with engineers	Reviewed wireframes; gathered suggestions for UI clarity (e.g., larger buttons for tablet use).	2
06-04-2025	Revising wireframes	Updated clickable wireframes based on engineer feedback.	3
08-04 to 12-04-2025	Writing TMA02 draft	Produced TMA02, incorporating domain analysis; tutor feedback later highlighted need for stronger academic focus.	10
15-04 to 25-04-2025	SOAR analysis and restructuring approach	Conducted SOAR analysis; decided to prioritise academic methodology over immediate commercial needs.	6

26-04 to 01-05-2025	Revisiting domain modelling and business rules	Refined conceptual classes; removed unnecessary classes (InspectionParameter, Reading).	6
02-05-2025	Finalising conceptual class diagram	Added corrected multiplicities; validated with engineers.	2
03-05-2025	Creating activity diagrams	Modelled engineer and manager workflows; two swimlanes to represent distinct roles.	3
04-05-2025	Tutor consultation	Discussed improved domain modelling; positive feedback.	1
05-05 to 09-05-2025	Writing Requirements Engineering section	Created use case diagrams and elaborated use cases; cross-referenced with business rules.	8
10-05 to 15-05-2025	Writing TMA03 draft	Focused on academic alignment; structured narrative per Sommerville's recommendations.	12
16-05-2025	Participant consent and ethical preparation	Prepared Participant Information Sheet and Consent Form; reviewed by tutor.	2
20-05-2025	Signing of consent forms	Engineers (including Joe Lane) signed consent to provide usability feedback.	1
22-05 to 02-06-2025	React Native research & prototype setup	Installed Expo Go; explored RN documentation; planned navigation structure.	6
03-06 to 12-06-2025	Developing Login and Sign-up screens	Built Login and Sign-up screens; tested on Android tablet.	12
13-06 to 18-06-2025	Creating Main Menu and Facilities Menu screens	Added navigation between screens; basic layout for facilities checklists.	10
19-06-2025	Usability testing with engineers	Engineers tested early prototype; positive feedback on clarity; suggestions recorded.	2
20-06 to 30-06-2025	Writing Implementation section	Documented code and design reasoning; explained iterative prototype development.	6
01-07 to 10-07-2025	Writing LSEPI & EDI section	Reflected on data protection, GDPR, and ethical participant involvement.	4
11-07 to 18-07-2025	Refining TMA03 draft	Edited personal reflection and requirements traceability; added updated domain diagram.	8
19-07 to 22-07-2025	Final proofread & updates	Integrated final tutor recommendations, updated appendices and project log.	6

Appendix 3 – Stakeholders' input

INITIAL MEETING TO DISCUSS FEASIBILITY OF BUILDING
“Routine Check Maintenance Application”

17 Feb 2025 17:00-18:00

Unit 20 IT Office

Attendees: Lukasz Brzozowski (LB)
 IT Manager (ITM)

AGENDA

- 1** Background of project proposal and problems addressed
- 2** Benefits to the organisation
- 3** Discussion – including practicalities and feasibility
- 4** Meeting outcomes/decisions/actions

1. Background of the project proposal and problems addressed.

Project background

My proposal is an application that assists the user with completing necessary maintenance routine documentation such as "Monthly Safety Checks", "Planned Maintenances", "Facilities Checks", amongst others. To provide paperless, time and storage saving solutions and increase the department's overall efficiency.

Problem areas the project will address:

- **limited access and storage capabilities**

Paper documents take up more physical storage space as they must be stored in file cabinets, which takes up additional space in the office that could be used by other means.

- **the inefficient process to complete the documentation**

Completing the paperwork takes excessive time due to printing, filing, scanning and the necessity of confirming the documents by the Plant and Services Manager, later called PSM.

- **limited management control**

Paper documents are separated for each monthly test and cannot be analysed without excessive extra time taken to do so. The Plant and Service Manager usually does this amongst other tasks, and for such a small Maintenance department, processing this documentation requires PSM to stay over his office hours or take this paperwork home.

There is also limited availability to the paperwork, as proved by the working-from-home scheme - could not access the documentation off-site.

- **security and clarity**

Some paper documents require handwriting, which can sometimes be hard to read, and paper forms can easily be lost or discarded in a production environment.

We had countless examples of lost paperwork over the years, which needed re-writing and stocking in folder cabinets to keep all the records and data intact for insurance purposes.

2. Benefits of the project to the organisation will include:

- Universal accessibility to allow greater control and management of the documentation
- Provision of digital storage to save physical storage space
- automated functionalities to save time when completing the documentation
- Increased safety and security of the data
- reduced reliance on paper documents, which supports the organisation's environmental strategy

3. Discussion:

Practicalities and feasibility of the project:

Practicalities:

LB: The application must be developed in a way that gives a reasonable control to the user.

LB: The User should be able to add, remove, edit, and sign the document digitally.

ITM: The User won't be able to sign the document digitally, as there is no facility to provide that. The only way to overcome this is to introduce user authorisation by providing a login and password to each user. Then, the name and surname of the person logged to the application can be added at the end of each routine check. Providing "digital sign".

LB: The application must be accessible from various devices.

ITM: The application can "shadow" on mobile devices as a Windows-based application, but we don't have the facility and skillset to develop a mobile application on Android or Apple operating systems. This is what other business-based applications do.

LB: The customer will have full control over the documentation produced.

ITM: We must also provide an admin facility, so IT have control over the application for security reasons.

LB: The Functionality of the application will be part manual and part automated

ITM: We can introduce period analysis; risk analysis and other analytics features to the application at later stages of the development.

- The customer will receive training after the completion

4. Meeting outcomes/decisions/actions.

1) Gather all the necessary information, including examples of the test sheets

Talk to all maintenance members who are doing the routine check sheets to gain more understanding about the expectations.

2) Work out how collected information could be represented in a data form and also how to make the test sheets generic (not hard-coded but dynamically changeable).

3) Create a forum or maybe email group where all information can be shared between the Maintenance team members who will be my customers and myself as a Project Development Lead.

4) Consider technologies you want to use to build the project. The company uses VB.NET codebase, and the majority of applications are written in that language, with applications being Microsoft Windows-based and running on the company's server.

That said, as Microsoft has gradually ceased support for VB.NET-based applications, the company will be transitioning their systems to yet another Windows-based programming language with long-term support – C#.

Appendix 4 – Glossary of domain terms

Term	Meaning
Inspection	Routine check performed on factory equipment to verify it is operating safely and correctly. A single form for an item to be inspected.
Engineer	A user responsible for conducting inspections and submitting reports.
Manager	A user who reviews inspection results and generates reports.
Subcheck	Supplementary sub-check that the engineer must perform during an inspection of a single item. There could be multiple sub-checks assigned to a single inspected item.

Result	A pass/fail result for a single inspection.
Item	A piece of equipment or machinery is being inspected.
Site	A specific physical location that could have a company name, building number, address, etc.
Zone	A specific area of the physical location within a Site.
Attachment	Supplementary piece of evidence that can be attached as part of the inspection form, e.g. image or document

Table 4.1.1 Glossary of domain terms

Appendix 4 – PROMPT Checklist

Below is a PROMPT checklist that includes categories and my reflections on the accounts of related literature used to support the project work.

Reading Source	PROMPT criteria					
	Presentation	Relevance	Objectivity	Method: for research reports	Provenance	Timeliness
	Is the information presented and communicated clearly? Consider the language, layout and structure.	Is the article relevant to the topic you are researching? Look at the introduction or overview to find out what it is mainly about.	Is the article biased, or motivated by a particular agenda? Is the language emotive? Are there hidden, vested interests?	Is it clear how the data was collected? Were the methods appropriate and can you trust it?	Is it clear where the information has come from? Can you identify the author(s), or organisation(s), and are they trustworthy? Are there references/citations that lead to further reading, and are they trustworthy sources?	How up to date is the material? Is it clear when it was written? Does the date of writing meet your requirements, or would it be obsolete?
<i>Fundamentals of Database Systems, Elmasri & Navathe (2021)</i>	This book helped me justify why I chose a relational database for my project. It	I used its explanations to back up my design choices for data storage and schema	It's an academic textbook that explains database theory and	It uses diagrams and structured examples to explain how relational models work.	A widely used textbook in database courses and cited in TM352.	Published in 2021, so it's recent and very applicable to current technologies.

	explained how relational models support data integrity and structure.	modelling. Which are highly relevant to the project.	practice without bias.	These aligned closely with my own system's needs.		
<i>Learning React Native: Building Native Mobile Apps with JavaScript, Eisenman (2019)</i>	The book and principles that it entails helped me with building my prototype UI in React Native before moving on to different tech stack. The book is written in simple language for beginners to understand. Additionally, it helped me understand the React Native whilst studying the TM352 module.	Useful for early prototyping. It assisted me with a rapid development of a front-end that could be shown to stakeholders.	It's a technical guide with no clear bias, just aimed at helping developers learn the framework.	Includes code examples and walkthroughs that I followed closely in my prototyping stage. It helped me to understand using the components and classes responsible for highlighting buttons and building more interactive user interface.	The book was written by Bonnie Eisenman, a respected and experienced software engineer that worked for brands like Google, Twitter and Codecademy. She is a web and mobile development expert with accounts of other books published by O'Reilly Media which is a respected publisher amongst software developers.	Published in 2017, as a second edition which may seem a little outdated, however the principles are still applicable to current technologies. There may be some additional reading required available via React Native official documentation, but the material is still relevant.
<i>A Comparative Study of Cross-platform Mobile Application Development Dongliang Y., Minjie H., (2021).</i>	This article helped me understand the pros and cons of using cross-platform tools like React Native, especially for quick minimum viable products (MVPs)	It supported my argument that React Native is suitable for prototyping but not ideal for final deployment in my case.	It's a peer-reviewed academic article comparing tools – so it's fair and evidence-based.	It used empirical testing to compare frameworks, which gave me confidence in applying its conclusions.	It's a peer reviewed conference paper that was presented at the Conference of Computing and IT Research and Education in New Zealand by respected PhD Senior Lecturer from the Wellington Institute of	The paper is dated at July 2021 and had been cited by many other university graduates as the ResearchGate data suggests.

					Technology – Minjie Hu.	
<i>Expo Documentation, (Expo, 2025)</i>	Clear examples and quick-start guides helped me deploy prototypes rapidly, increasing stakeholder engagement.	Highly relevant for speeding up feasibility testing. It enabled me to validate UI workflows before committing to full development.	Although produced by the Expo team, its factual content was reliable for practical prototyping tasks.	Following official instructions built my confidence in working with technical documentation, a valuable professional skill.	As the official source, it ensured I was using best practices recognised by the React Native community.	Its 2025 updates meant I was working with the latest stable methods, reducing debugging issues during early testing.
<i>Software Engineering Sommerville (2016)</i>	This book helped me understand the value of iterative development and requirements engineering. It gave me a foundation for planning the lifecycle of my project.	Very relevant since I used many concepts from this book. When I justified my use of mixture of waterfall and iterative development, designed UML diagrams, completed domain modelling and continued to work on other SDLC principles.	It's a well-respected academic source and doesn't try to sell a product. Instead, it's focused on explaining best practices relevant to software development life cycle.	The book explains software engineering principles through examples and case studies. I partially replicated some of its methods, but most important part is the step-by-step process from problem identification to solution creation helped me reflect on my own choices.	Ian Sommerville is a well-known professor in software engineering, and the principles he covers are very relevant to the industry. Besides the material is widely cited within the TM354 module books and was additionally recommended by my TM354 module tutor during my studies.	Published in 2016 – it's not brand new, but the practices are still standard in industry and relevant for my work. Software engineering series written by Ian Sommerville are also regularly updated with new discoveries from the field.
<i>Modern Web Development with ASP. NET Core 3, Peres, Ricardo (2020)</i>	This source gave me insight into how enterprise systems use .NET, which matches the tech stack required by the organisation.	Relevant – I used this to compare the tech stack with React Native and inspect how big is my learning curve if I was to implement the system in .NET using C#.	Although the author promotes Microsoft technologies, the book is still informative and aligns with real enterprise practices. The downside is that it covers ASP.NET technology which I am not	The book includes architectural examples that are practical. I should be able to refer to these examples when designing my own application's layout.	The author is a Microsoft MVP and expert in .NET development, which adds credibility.	Published by O'Reilly, which is known for quality developer books.

			very familiar with.			
<i>Creating Mobile Apps with Xamarin.Forms, Petzhold (2016)</i>	Step-by-step tutorials with screenshots; practical rather than theoretical. The tutorials follow step-by-step process confirmed by the Microsoft. It can be used as a reference book.	Relevant mainly for comparing Xamarin.Forms with React Native which helped explain why Xamarin was unsuitable for rapid prototyping.	The book forms an objective and technical manual. As a downside it is not academically rigorous. But on a positive note, the material is unbiased.	Provides hands-on examples, though lacking formal evaluation or practical testing.	Written by a Xamarin expert and published by Microsoft Press, giving credibility.	Dated at 2016 Xamarin.Forms is now being replaced by .NET MAUI as confirmed by the Microsoft, which is limiting Xamarin. Forms direct applicability.
<i>Building Xamarin.Forms Mobile Apps Using XAML: Mobile Cross-Platform XAML and Xamarin.Forms Fundamentals, Dan Hermes, Nima Mazloumi (2019)</i>	Tutorial-focused, allows a brief understanding how XAML could structure UI layouts. The book was mostly useful for comparison with other technologies since Xamarin is being replaced with .NET MAUI	Relevant only for background research; confirmed my decision to avoid Xamarin for this project.	Objective and unbiased, but clearly not intended for academic analysis.	Reinforced my learning about UI layout structures, though I treated it as secondary evidence.	Written by Dan Hermes a .NET and Xamarin expert from is a Microsoft Regional Director, Xamarin MVP, Microsoft MVP, IBM Champion, and founder of Lexicon Systems. supported by Dr Nima Mazloumi a Senior Director for One Planet Ops Inc with over 18 years of experience in full-stack development. and Glen Stevens, a software developer with over 30 years of software development experience. Currently	Although Xamarin is superseded by .NET MAUI many developers use Xamarin. Forms in their mobile applications.

					working with mobile and cloud technologies.	
<i>Introducing .NET MAUI: Build and Deploy Cross-platform Applications Using C# and .NET Multi-platform App UI, Lawrence S. (2023)</i>	Although the book's targeted audience are intermediate and advanced level developer the explanations of .NET MAUI features, including camera integration are written in a non-overly complicated language.	Relevant for evaluating MAUI as a potential long-term cross-platform solution.	Some promotional bias towards Microsoft, but still technically very informative.	Practical examples helped me compare MAUI's capabilities against Windows-based apps. And although photo taking would be supported via the native platform library working in cooperation with .NET MAUI its implementation seems a little complicated	Written by a known Microsoft trainer; good technical credibility.	Very recent; confirmed MAUI is future-proof but not suitable under current organisational policies.
<i>Master C# and SQL by Building Applications, Packt (2022)</i>	A course that includes video material as well as support text. Practical examples easy to follow for linking C# with SQL Server.	Relevant for implementation skills rather than academic justification.	Not an academic source but focuses well on practical training.	Includes hands-on exercises which could build my confidence in C# database integration.	Moderate credibility; Packt is known for developer-oriented books.	Recent enough to remain relevant for Windows desktop development.
<i>Mobile Design Pattern Gallery, 2nd Edition, Neil Theresa (2014)</i>	This book gives insight to various navigation and user interface design patterns used in mobile devices. It's comprised of various visual patterns, images, examples of mobile screen layouts, and	Relevant to user interface design of the application, considers mobile forms and navigation and these two are crucial for the project UI.	Not an academic source but objective and unbiased practical guide to design patterns for smartphone apps	Includes practical guides and examples which patterns are most suited for which type of applications, very relevant to digital forms applications as it has a chapter about designing forms.	Written by a user experience consultant who designed applications for start-ups and Fortune500 companies.	Although published nearly 10 years ago it is very much relevant nowadays.

	other images to allow better understanding of the design process and decision making for each pattern.					
--	--	--	--	--	--	--

Appendix 5 – Evolution of classes, attributes and domain structural models

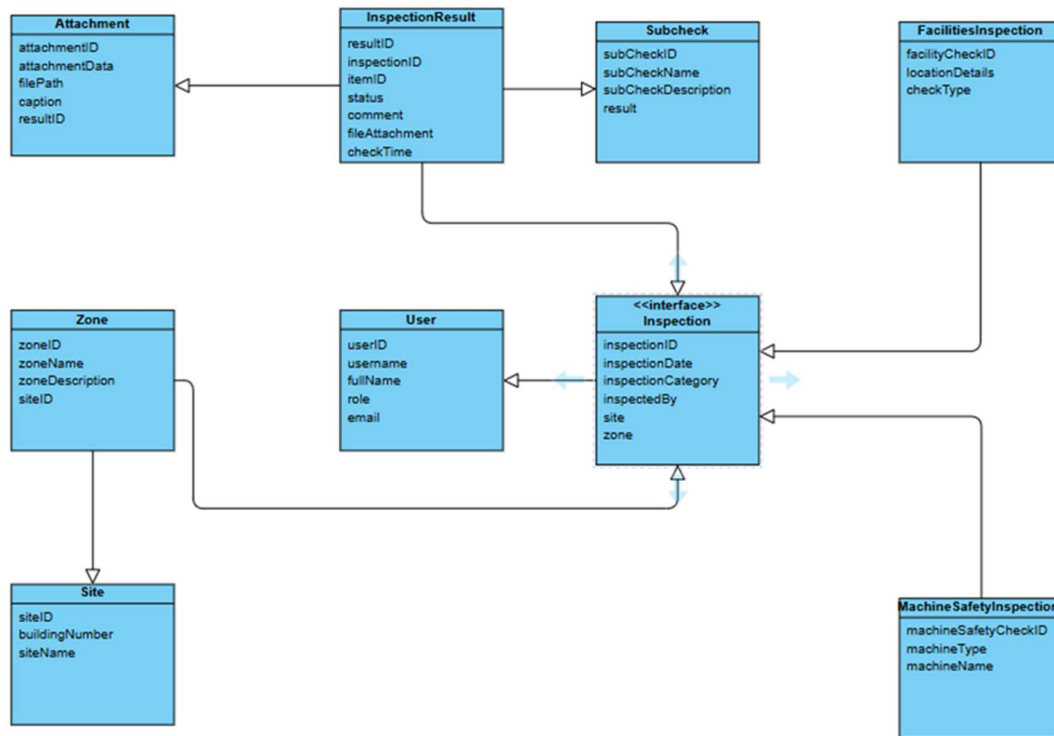
Initial version of conceptual classes and attributes:

Class name	Description	Attributes
User	Represents an individual interacting with the system, either as an engineer performing inspections or a manager reviewing outcomes. Each user is automatically identified on login, enabling audit trails and role-based access control throughout the application.	userID username fullName role email
FacilitiesInspection	Serves as the base class for the facility inspection form. It captures generic details such as inspection date, inspector identity, inspected item location details (zone and site), inspected item name and type.	facilityInspID inspectionDate inspectedBy -> User zone -> Zone site -> Site itemType (e.g Emergency Lighting) itemName (e.g. Twin spot emergency light)
MachineSafetyInspection	Serves as the base class for a machine safety inspection form. It captures generic	machineSafetyInspID inspectionDate

	details such as inspection date, inspector identity, inspected item location details (zone and site), inspected item name and type.	inspectedBy -> User zone -> Zone site -> Site itemType (e.g Die-Cut machine, Finishing machine) itemName (e.g. Bobst, Supernova)
Subcheck	This class represents the subcomponents of a single inspection, like "Test E stop," "Inspect guards," or "Check light fitting". So, in other words, it represents what is being checked across any inspection.	itemID: unique ID for the individual inspection item (sub-check) itemName: short label (e.g. "E-Stop", "Door") itemDescription: optional full description valueType: defines expected input type (e.g., "boolean", "number", "status") passCriteria: String – logic to evaluate pass/fail
InspectionResult	Stores the actual outcome for a given item during an inspection. Includes values like Pass/Fail, timestamp, optional comments, and attached evidence (e.g., photos).	resultID: unique result ID inspectionID -> Inspection (Foreign Key to Inspection) itemID -> InspectionSubCheck (refers to InspectionSubCheck) status: Boolean (e.g., actual outcome: Pass/ Fail) comment: optional inspector comment photoAttachment: String (file path or URL) - optional photo link checkTime: timestamp of check (optional for audit readiness, multiple checks tracking, and analytics)

Site	Represents the buildings or sites being inspected (e.g. Unit 6, Unit 22). Useful for organising inspections by physical location or reporting at the site level.	siteID: unique ID buildingNumber (e.g. "U20", "U22", "U6") siteName (Packaging, Conductive, Protective)
Zone	Sub-location within a unit. This helps narrow down item locations for inspections (e.g., Rear, Front, Right Side, etc.).	zoneID: unique zone ID zoneName: Zone label zoneDescription: additional zone description siteID: Foreign Key to Site
Attachment	Supports additional files, including photos for visual evidence of any item check, and document files for additional information. The attached photos could be used for an audit or a fault investigation.	attachedID: Unique ID fileData: filePath: Storage reference caption: Optional note resultID: related InspectionResult

And its associated domain structural model:



Proposed Multiplicities:

User – Inspection

- One user (engineer or manager) can create or complete multiple inspections, but a specific inspection is assigned to one specific user

User (1) ----- (*) Inspection

Inspection – Subcheck

- Each inspection consists of many subchecks, but each subcheck is linked to exactly one specific inspection.

Inspection (1) ----- (*) Subcheck

Subcheck – Result

- Each subcheck produces exactly one inspection result per inspection, but a result is tied to multiple subchecks

Subcheck (*) ----- (1) Result

Result – Attachment

- One inspection result can have zero or more attachments, but each attachment belongs to exactly one result.

Result (1) ----- (0..*) Attachment

Inspection – Site

- Each inspection takes place at exactly one site, but each site can have many inspections over time.

Site (1) ---- (*) Inspection

Site – Zone

- Each site consists of one or more zones, but each zone belongs to exactly one site.

Site (1) ---- (1..*) Zone

Inspection – Zone

- Each inspection occurs in exactly one zone, but each zone can have many inspections.

Zone (1) -- (*) Inspection

Inspection – Result

- Each inspection can have only one result, but one Result can be assigned to many inspections.

Inspection (1) ---- (*) Result

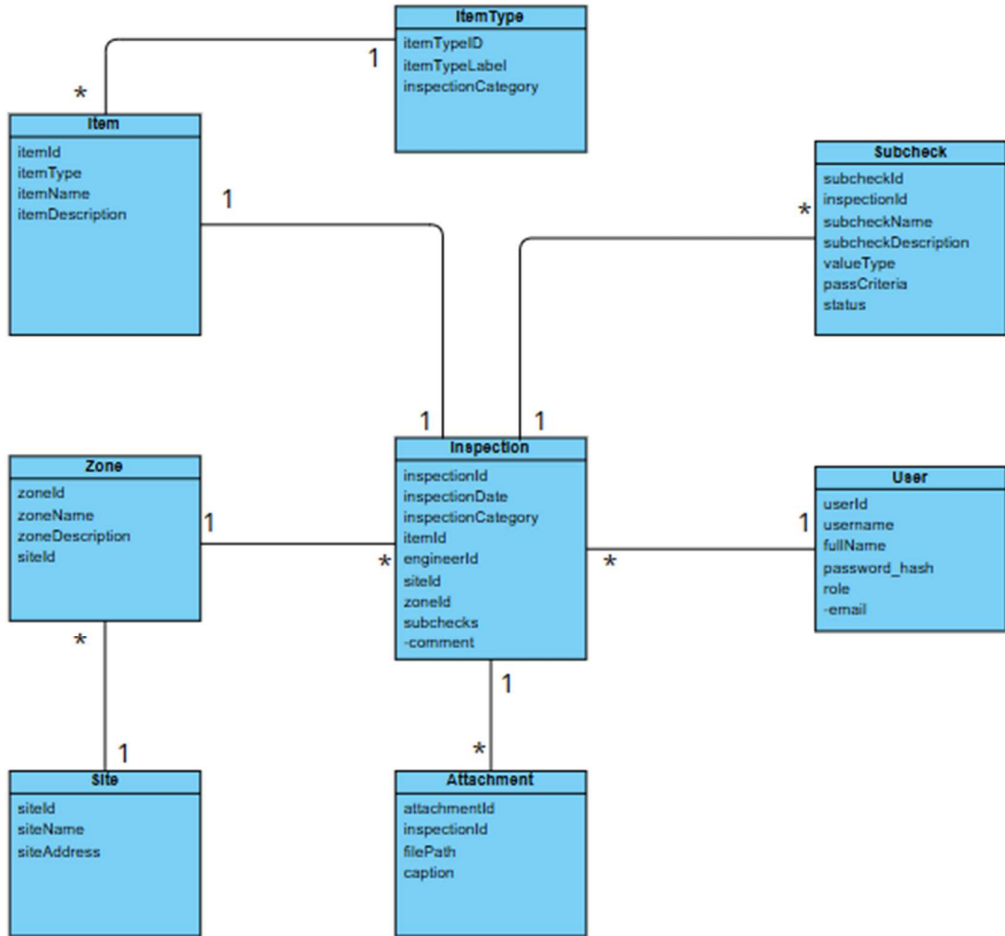
Evolved version of the conceptual classes and attributes:

Class name	Description	Attributes
User	Represents an individual interacting with the system, either as an engineer performing inspections or a manager reviewing outcomes. Each user is automatically identified on login, enabling audit trails and role-based access control throughout the application.	userID: integer; username: string; fullName: string; password_hash: string; role: "Engineer", "Manager", "Admin" email: string
Inspection	Serves as the base class for all inspections. It captures generic details such as inspection date, inspector identity, and the category of inspection. All specific inspections (e.g., facility or machine-based) inherit from this. Having both categories, facility and machine safety, under one common inspection class allows a more generic and dynamic way of storing information.	inspectionID: integer; inspectionDate: string; inspectionCategory: string (Facilities/MachineSafety) itemID: integer -> Item ID inspectorId: integer -> User ID siteId: integer -> Site ID zoneId: integer -> Zone ID Subchecks[] -> Subcheck list of Subcheck items comment: string

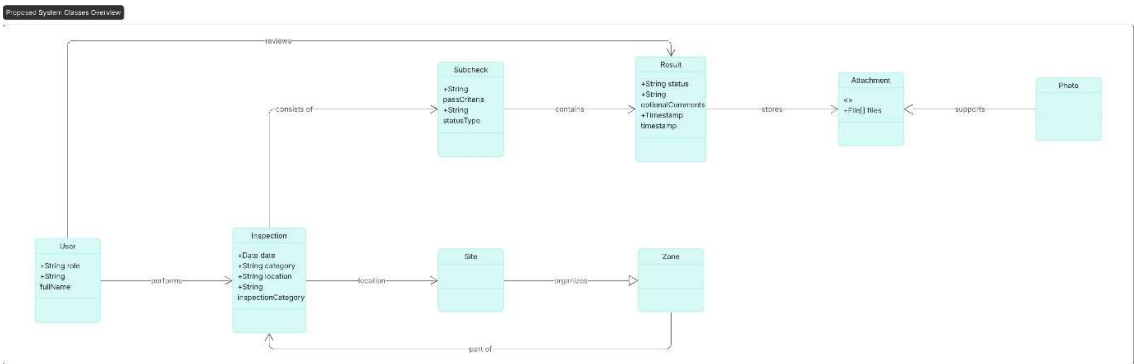
Subcheck	This class represents the subcomponents of a single inspection, like "Test E stop," "Inspect guards," or "Check light fitting". So, in other words, it represents what is being checked across any inspection.	subcheckID: integer - unique ID for the individual inspection item (sub-check) inspectionID: integer -> points to Inspection subcheckName: string - short label (e.g. "E-Stop", "Door") subcheckDescription: string - optional, full description valueType: defines expected input type (e.g., "boolean", "integer", "string") passCriteria: String – logic to evaluate pass/fail (e.g. "true", ">=50", "GREEN") status: String (optional) – for status-based categories e.g. ("pass", "fail" or "na")
Site	Represents the buildings or sites being inspected (e.g. Unit 6, Unit 22). Useful for organising inspections by physical location or reporting at the site level.	siteID: unique ID siteName (Packaging, Conductive, Protective) siteAddress: string
Zone	Sub-location within a unit. This helps narrow down item locations for inspections (e.g., Rear, Front, Right Side, etc.).	zoneID: unique zone ID zoneName: Zone label zoneDescription: additional zone description siteID: Foreign Key to Site
Item	Supports a piece of equipment that is being inspected, e.g, a machine or emergency light	itemId: number; itemType: string; // "Emergency Lighting" "Die-Cut" etc.

		itemName: string; // "EL-01" "Bobst DC-01" description: string; zoneId: number;
Attachment	Supports additional files, including photos for visual evidence of any item check, and document files for additional information. The attached photos could be used for an audit or a fault investigation.	attachedID: Unique ID fileData: filePath: Storage reference caption: Optional note

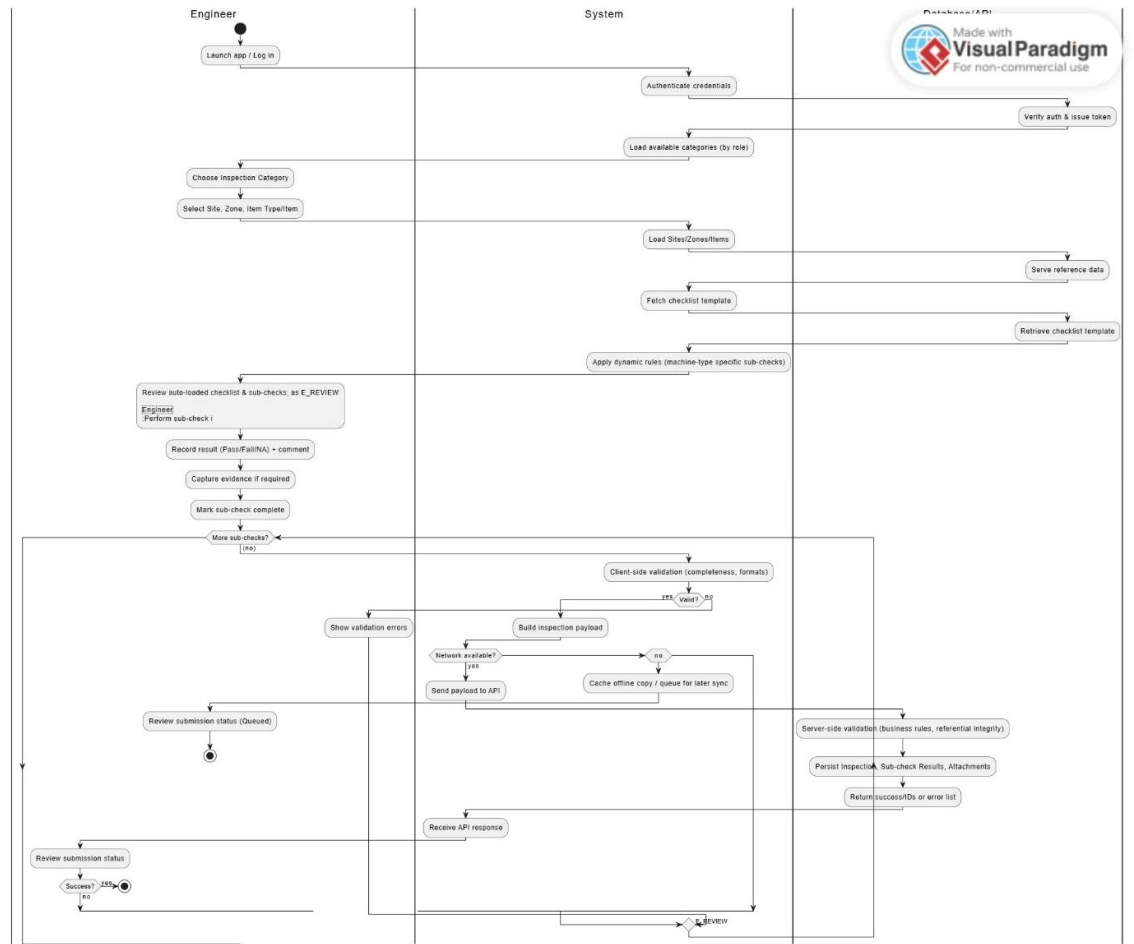
And its associated domain structural model:



Appendix 6 – Supplementary system diagrams



Basic system class diagram



Activity diagram: Completing an inspection form

Appendix 7 – Elaborated use cases

Use case: Complete an Inspection (Engineer)

<i>Use Case ID</i>	<i>UC-001</i>
<i>Actors</i>	Engineer
<i>Description</i>	The engineer completes a scheduled inspection by filling out digital forms, marking each sub-check as pass/fail, adding comments, and attaching photos where required.
<i>Preconditions</i>	The engineer is logged in and has been assigned at least one active inspection.
<i>Basic Flow</i>	1. Engineer selects the scheduled inspection from the list. 2. System loads the corresponding checklist (subchecks). 3. Engineer inspects each item and records a pass/fail result. 4. For failed items, the engineer adds comments and attaches at least one photo. 5. Engineer submits the inspection.
<i>Alternative Flows</i>	AF1: Engineer cancels the inspection before submission – system saves it as “In Progress.” AF2: System prevents submission if required evidence (comments/photos) is missing for failed checks (BR2).
<i>Postconditions</i>	Inspection status updated to “Submitted” and visible to the manager for review.

Figure 4.2.4 Elaborated use case for an engineer completing an inspection form

Use case: Log in (Engineer and Manager)

<i>Use Case ID</i>	<i>UC-003</i>
<i>Actors</i>	Engineer, Manager

Description	The user inserts their credentials and is being authorised to access the application.
Preconditions	The user inserts correct credentials
Basic Flow	1. The user inserts their correct login and password 2. System sends a request to the database to check the user credentials 3. System loads the application main screen 4. User is permitted to create the form, or use template
Alternative Flows	AF1: User cancels the login and receives the prompt that login process was cancelled AF2: System prevents login if the credentials are incorrect
Postconditions	User is authorised to access the application functionalities.

Table 4.2.3 Elaborated use case for user log-in functionality

Use case: Review an Inspection (Manager)

Use Case ID	UC-002
Actors	Manager
Description	The manager reviews submitted inspections, approves them, and assigns follow-up tasks if needed.
Preconditions	At least one inspection has been submitted.
Basic Flow	1. Manager logs in and selects a submitted inspection. 2. System displays inspection results and attachments. 3. Manager reviews failed items and confirms or rejects the inspection. 4. Manager assigns corrective actions if necessary.
Alternative Flows	AF1: Manager rejects the inspection – system notifies the assigned engineer to revise and resubmit.
Postconditions	Inspection status updated to “Approved” or “Rejected” and archived for compliance.

Appendix 8 – Functional and non-functional requirements

FR1 – User Management

FR ID	Requirement	MoSCoW priority
FR 1.1	The system shall allow users (engineers, managers, admins) to log in using a username and password. (Must)	Must
FR 1.2	The system shall allow engineers to create an account with their full name, email, and password. (Must)	Must
FR 1.3	The system shall identify the logged-in user and display their name and role in the application header. (Must)	Must
FR 1.4	The system shall restrict functionality based on user roles (e.g. managers can review reports, engineers can only submit inspections)	Should

Table 4.2.5: FR1 - Functional requirements for user management

FR 2 - Site and location selection

FR ID	Requirement	MoSCoW priority
FR 2.1	The system shall allow engineers to select or insert a site (e.g., "Unit 20").	Must
FR 2.2	The system shall allow engineers to select or insert a location (e.g., "First Floor", "Rear") within the chosen site	Must
FR 2.3	The system shall ensure only zones belonging to the selected site are displayed.	Must

Table 4.2.6: FR2 - Functional requirements for site and location selection

FR 3 – Inspection Type selection

FR ID	Requirement	MoSCoW priority
FR 3.1	The system shall allow engineers to select an inspection category: • Facility Check (e.g. Emergency Lights, Fire Doors) • Machine Safety Check (e.g. Die-Cut Machine, Print Machine)	Must

FR 3.2	The system shall dynamically load available inspection types based on the selected category.	Must
--------	--	------

Table 4.2.6: FR3 – Functional requirements for inspection type selection

FR 4- Performing an Inspection

FR ID	Requirement	MoSCoW priority
FR 4.1	The system shall load the appropriate inspection form based on the selected inspection type.	Must
FR 4.2	The system shall display subchecks dynamically, based on machine type or facility type.	Must
FR 4.3	The system shall allow the engineer to record outcomes for each subcheck: • Boolean (Pass/Fail) (Must) • Numeric (e.g. readings, measurements) (Should)	Must
FR 4.4	The system shall allow the engineer to add an optional comment for each subcheck.	Must
FR 4.5	The system shall allow attaching optional evidence (Attachment class; as per the Business Rule 3)	Must
FR 4.6	The system shall automatically calculate the overall inspection result (Pass only if all mandatory sub checks pass).	Must
FR 4.7	The system shall allow the engineer to submit and save a completed inspection.	Must

Table 4.2.7: FR4 – Functional requirements for completing an inspection

FR 5 – Inspection review and reporting

The prototype will skip this, but I included it for completeness and future scope.

FR ID	Requirement	MoSCoW priority
FR 5.1	The system shall allow managers to view all completed inspections filtered by date, site, or engineer.	Should

FR 5.2	The system shall allow managers to view inspection details, including comments and photo evidence.	Should
FR 5.3	The system shall allow exporting inspection results (PDF or Excel).	Could

Table 4.2.8: FR5 – Functional requirements for inspection review and reporting

FR 6 – Notifications

This is yet another future optional functionality of the application. It will be omitted in this prototype.

FR ID	Requirement	MoSCoW priority
FR 6.1	The system shall notify managers when a critical inspection fails.	Could
FR 6.2	The system shall allow engineers to view system notifications (e.g., scheduled inspections, overdue checks)	Could

Table 4.2.9: FR5 – Functional requirements for displaying notifications

2. Non-functional requirements

Although the prototype application will only consider the usability, security and partial maintainability of the application, I listed most of the industry-standard non-functional requirements in the categorised tables.

NFR 1 – Usability

NFR ID	Requirement	MoSCoW priority
NFR 1.1	The system shall provide a tablet-friendly interface with touchscreen-optimised controls.	Must
NFR 1.2	The system shall use clear labels and controls.	Must

Table 4.2.10: NFR1 – Non-functional requirements for usability

NFR 2 – Performance

NFR ID	Requirement	MoSCoW priority
NFR 2.1	Forms shall load in under 3 seconds.	Should

NFR 2.2	Saving an inspection shall take no more than 2 seconds.	Should
---------	---	--------

Table 4.2.11: NFR2 – Non-functional requirements for performance

NFR 3 – Security

NFR ID	Requirement	MoSCoW priority
NFR 3.1	Only authenticated users shall access the system.	Must
NFR 3.2	Data shall be encrypted during transmission.	Should

Table 4.2.12: NFR3 – Non-functional requirements for security

NFR 4 – Reliability

NFR ID	Requirement	MoSCoW priority
NFR 4.1	The system shall prevent data loss if a device loses connection (local saving)	Should

Table 4.2.13: NFR4 – Non-functional requirements for reliability

NFR 5 – Maintainability

NFR ID	Requirement	MoSCoW priority
NFR 5.1	The system shall allow new inspection types and subchecks to be added without code changes (dynamic from a database)	Should
NFR 5.2	The system programming language and technology used will meet the organisation's software policies.	Could

Table 4.2.14: NFR5 – Non-functional requirements for maintainability

Appendix 9 – Wireframes – screenshots



Back

Facility Check

Add New Check

Air Conditioning Check

BIO Waste Treatment Plant Check

Emergency Lights Test

Fire Exits Inspection

Fire Extinguisher Inspection

Fire Test

Ladder Log

Legionella Check

Pest Control Check

Racking Inspection



Back

Emergency Light Monthly Check

Date

date Type

Location

Canteen, Warehouse...

Zone

A,B,...

Switch ID

A, B, C, etc...

Light ID

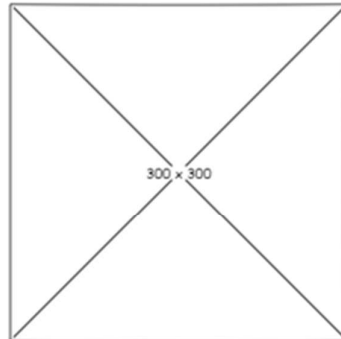
A1, A3, F9...

Light Type

Twin Spot, Square, Bulkhead...

Additional Notes

Image



☐ Pass

☐ Fail

Completed by:

User's Full Name

Cancel

Save



Back

New Machine Monthly Safety Check

Date

16/06/2024

Machine

Bobst...

Machine Group

Die Cut...

Choose completed safety tests

- ☐ Function test all e-stops
- ☐ Function test all light guards
- ☐ Function test gate switches
- ☐ Function master panel key
- ☐ Function test pull cords

Comments

The LHS rear e-stop had loose button.

Completed by:

User's Full Name

Cancel

Save



Maintenance Forms App

User's Full Name



Notification

Your request for a new account has been sent to the Line Manager for approval.
You will receive an email once your account is approved and created.

OK

Appendix 10 – Database code and tables

```
PRAGMA foreign_keys = ON;

-- USERS
CREATE TABLE IF NOT EXISTS users (
  user_id INTEGER PRIMARY KEY,
  username TEXT NOT NULL,
  full_name TEXT NOT NULL,
  password_hash TEXT NOT NULL,
  role TEXT NOT NULL CHECK (role IN ('Engineer','Manager','Admin')),
  email TEXT NOT NULL UNIQUE
);

-- SITES
CREATE TABLE IF NOT EXISTS sites (
  site_id INTEGER PRIMARY KEY,
  site_name TEXT NOT NULL,
  site_address TEXT
);

-- ZONES
CREATE TABLE IF NOT EXISTS zones (
  zone_id INTEGER PRIMARY KEY,
  zone_name TEXT NOT NULL,
  zone_description TEXT,
  site_id INTEGER NOT NULL,
  FOREIGN KEY (site_id) REFERENCES sites(site_id) ON DELETE CASCADE
);

-- ITEM TYPES (Used in templates)
CREATE TABLE IF NOT EXISTS item_types (
  item_type_id INTEGER PRIMARY KEY,
  item_type_label TEXT NOT NULL UNIQUE
  item_category TEXT NOT NULL CHECK (item_category IN ('Facility', 'Machine Safety'))
);
```

```
-- ITEMS
CREATE TABLE IF NOT EXISTS items (
    item_id INTEGER PRIMARY KEY,
    item_type TEXT NOT NULL,
    item_name TEXT NOT NULL,
    item_description TEXT,
    zone_id INTEGER NOT NULL,
    FOREIGN KEY (zone_id) REFERENCES zones(zone_id) ON DELETE CASCADE,
    FOREIGN KEY (item_type) REFERENCES item_types(item_type_label) ON DELETE CASCADE
);

-- INSPECTIONS
CREATE TABLE IF NOT EXISTS inspections (
    inspection_id INTEGER PRIMARY KEY,
    inspection_date TEXT NOT NULL,
    inspection_category TEXT NOT NULL CHECK (inspection_category IN ('Facility', 'Machine Safety')),
    item_id INTEGER NOT NULL,
    engineer_id INTEGER NOT NULL,
    site_id INTEGER NOT NULL,
    zone_id INTEGER NOT NULL,
    comment TEXT,
    FOREIGN KEY (item_id) REFERENCES items(item_id) ON DELETE RESTRICT,
    FOREIGN KEY (engineer_id) REFERENCES users(user_id) ON DELETE RESTRICT,
    FOREIGN KEY (site_id) REFERENCES sites(site_id) ON DELETE CASCADE,
    FOREIGN KEY (zone_id) REFERENCES zones(zone_id) ON DELETE CASCADE
);
```

```

-- SUBCHECK TEMPLATES
CREATE TABLE IF NOT EXISTS subcheck_templates (
    subcheck_template_id INTEGER PRIMARY KEY,
    item_type TEXT NOT NULL,
    subcheck_name TEXT NOT NULL,
    subcheck_description TEXT,
    value_type TEXT NOT NULL CHECK (value_type IN ('string','number','boolean')),
    pass_criteria TEXT,
    FOREIGN KEY (item_type) REFERENCES item_types(item_type_label)
);

-- SUBCHECK RESULTS
CREATE TABLE IF NOT EXISTS subchecks (
    subcheck_id INTEGER PRIMARY KEY,
    inspection_id INTEGER NOT NULL,
    subcheck_name TEXT NOT NULL,
    subcheck_description TEXT,
    value_type TEXT NOT NULL CHECK (value_type IN ('string','number','boolean')),
    pass_criteria TEXT,
    status TEXT NOT NULL CHECK (status IN ('pass', 'fail')),
    FOREIGN KEY (inspection_id) REFERENCES inspections(inspection_id) ON DELETE CASCADE
);

-- RESULTS (caching result per inspection)
CREATE TABLE IF NOT EXISTS results (
    result_id INTEGER PRIMARY KEY,
    inspection_id INTEGER NOT NULL UNIQUE,
    overall_status TEXT NOT NULL CHECK (overall_status IN ('pass', 'fail')),
    FOREIGN KEY (inspection_id) REFERENCES inspections(inspection_id) ON DELETE CASCADE
);

-- READINGS
CREATE TABLE IF NOT EXISTS readings (
    reading_id INTEGER PRIMARY KEY,
    inspection_id INTEGER NOT NULL,
    parameter_id INTEGER NOT NULL, -- e.g. subcheck_id or parameter template
    value TEXT NOT NULL,
    unit TEXT NOT NULL,
    FOREIGN KEY (inspection_id) REFERENCES inspections(inspection_id) ON DELETE CASCADE
);

-- ATTACHMENTS
CREATE TABLE IF NOT EXISTS attachments (
    attachment_id INTEGER PRIMARY KEY,
    inspection_id INTEGER NOT NULL,
    file_path TEXT NOT NULL,
    caption TEXT,
    FOREIGN KEY (inspection_id) REFERENCES inspections(inspection_id) ON DELETE CASCADE
);

```


Further Appendices